



REPORT ON FINAL PROJECT

Name : RANJANI M

Batch No : DA|TN-FNB03

Contact : +918754609051

Email ID : ranjani.mani22@gmail.com

Domain : Retail Sales & E-Commerce

Project Title : Adidas Sales Data Analysis Using Python & Power BI

Submission Date : 09-12-2025

Mentor : Mr.Kumaran

Data Source : Kaggle.com

Raw Dataset Link : [Raw Dataset.csv](#)

Cleaned Dataset Link: [Cleaned_Dataset.csv](#)

Adidas Sales Data Analysis Using Python and Power BI

Domain:

Retail Sales & E-Commerce

Objective:

- To clean and prepare the Adidas sales dataset by handling missing values, correcting incorrect totals, and ensuring appropriate data types.
- To understand how sales vary across regions, states, retailers, and product categories.
- To examine monthly and annual sales trends for the year 2021.
- To create meaningful visualisations that clearly display patterns and relationships within the dataset.
- To summarise the main findings and highlight insights that can support improved business and sales decision-making.

Outcome

- A fully cleaned and organised Adidas sales dataset ready for analysis.
- Visualisations that clearly show how sales vary across months, regions, retailers, and product categories.
- A better understanding of which areas and products performed well and which ones underperformed.
- Identification of important sales trends for the year 2021.
- A set of key insights explaining the main factors affecting Adidas sales, supporting better business and decision-making.

Dataset Information (Raw):

Source: Kaggle - Adidas US Sales Analysis(2022) - Available on Kaggle since around 2022

Year / Timeline: The dataset contains sales data from 2020 to 2021, with the majority of records belonging to 2021 and comparatively fewer entries from 2020.

Dataset Size: Total Rows: 5,000 & Total Columns: 12

Dataset Description: This dataset contains detailed sales transactions of Adidas products across the United States. Each entry represents a single sale and includes information such as the date of the transaction, the retailer involved, and the location (region, state, and city). The dataset also provides product details like the category of the item sold, price per unit, and the number of units sold.

Along with this, the dataset includes important financial fields such as total sales and operating profit, which help in analysing revenue and profitability. The sales method column shows whether the sale happened online, in-store, or through an outlet. Although the data covers both 2020 and 2021, most of the records belong to 2021.

Features: The dataset includes useful information such as:

- Retailer – Name of the store or platform where the product was sold.
- Retailer ID – Internal code for retailers; same retailer may appear under different IDs.
- Invoice Date – Date of the transaction.
- Region – Broad U.S. region (West, Midwest, South, etc.).
- State – U.S. state of the transaction.
- City – City where the sale occurred.
- Product – Product category sold (e.g., footwear, apparel).
- Price per Unit – Selling price of one unit.
- Units Sold – Quantity sold in the transaction.
- Total Sales – Total revenue generated.
- Operating Profit – Profit after subtracting operational costs.
- Sales Method – Type of sale: Online, In-Store, or Outlet.

Type of Analysis:

1. Descriptive Analysis (summarising the dataset)

This part focuses on understanding the basic structure and patterns in the dataset. I will summarise the sales data using statistics and visualisations to identify how sales vary across regions, retailers, product categories, and months. Trend analysis is also included to observe monthly and yearly sales patterns.

This helps provide a clear picture of the overall sales performance.

2. Diagnostic Analysis (finding reasons behind patterns)

After identifying the main patterns, I will explore why these patterns occur. For example, I will analyse why certain regions show higher sales, why some retailers perform better, or why specific months experience peak performance. This helps explain the reasons behind the trends observed in the descriptive analysis.

Problem Definition:

The main goal of this project is to understand how Adidas sales are happening across different parts of the United States. Even though we have the sales dataset, it doesn't directly tell us:

- which regions and cities are performing well,
- which products sell the most,
- how the price and number of units sold affect total sales, or
- whether online, in-store, or outlet sales do better,
- So, through this project, I want to explore the data properly and find out where the sales are high or low,
- which product categories are popular,
- how sales change based on method (online, outlet, in-store), and
- what patterns and trends are present in the overall sales performance.

In simple terms, the business problem here is to clearly understand how Adidas is performing across different locations, products, and sales channels using the data provided.

Expected Outcome:

After analysing the dataset, I expect to get a clear picture of how Adidas sales are distributed across the US. By the end of the analysis, I should know:

- which regions, cities, and retailers contribute more to sales,
- which product categories perform well and which do not,

- how the sales method affects revenue and profit,
- what trends appear across months or years, and
- what might be the reasons behind high or low sales in certain places.

Overall, the expected outcome is to get meaningful insights that explain what is happening in the sales data and understand why some areas, products, or sales methods perform better than others.

Import libraries, load dataset :

```
[2]
✓ 3s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("/content/drive/MyDrive/MAIN PROJECT_ENTRI/Raw Dataset.csv")
df
```

	Retailer	Retailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Sales Method
0	Foot Locker	1128299	2021-06-24	Southeast	Virginia	Richmond	Men's Street Footwear	\$53.00	233	1,235	\$580	Online
1	Sports Direct	1197831	2021-08-18	South	Alabama	Birmingham	Women's Apparel	\$26.00	120	312	\$206	Outlet
2	Amazon	1185732	2021-06-11	Midwest	Ohio	Columbus	Women's Apparel	\$64.00	196	1,254	\$778	Online
3	Foot Locker	1128299	2021-10-22	West	Arizona	Phoenix	Women's Street	\$45.00	325	14,625	\$5,119	Online

Variables Terminal 09:58 Python 3

To start the analysis, I first imported the essential Python libraries required for working with the data. These libraries help in performing different tasks such as reading the dataset, cleaning values, analysing patterns, and creating visualisations.

After importing the libraries, I loaded the Adidas sales dataset into a pandas DataFrame. This allowed me to view the data in a structured table format. I checked the first few rows to ensure the dataset was successfully loaded and that all columns appeared correctly.

Dataset description (rows, columns, features):

```
Dataset description (rows, columns, features)

[3]
✓ 0s
rows = df.shape[0] #no. of rows
columns = df.shape[1] #no. of columns

print("Rows:", rows)
print("Columns:", columns)
```

Rows: 5000
Columns: 12

```
[4]
✓ 0s
df.columns #column names

Index(['Retailer', 'Retailer ID', 'Invoice Date', 'Region', 'State', 'City',
      'Product', 'Price per Unit', 'Units Sold', 'Total Sales',
      'Operating Profit', 'Sales Method'],
      dtype='object')
```

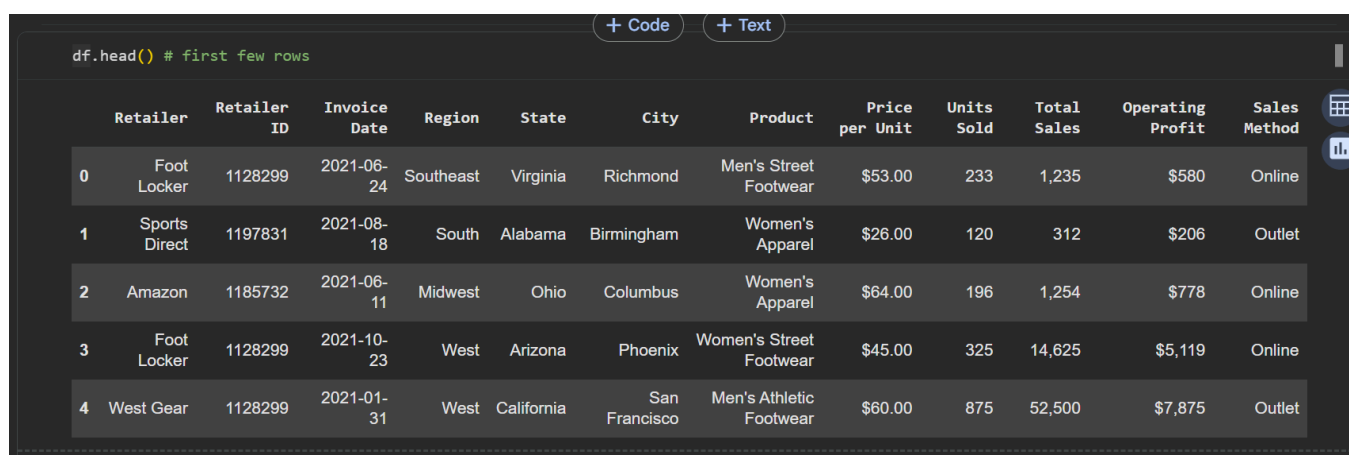
After loading the dataset, I checked the total number of rows and columns to understand the overall size of the data. The output confirmed that the dataset contains **5,000 rows** and **12 columns**, which means there are 5,000 individual sales records and 12 different attributes describing each record.

Next, I displayed the list of **column names** to understand what features are included in the dataset. These columns represent important information such as the **retailer, date of sale, location details (region, state, city), product category, pricing, total sales amount, operating profit, and the method of sale.**

This step helps in getting a clear overview of what data is available and how each column will be used in the analysis.

Initial EDA before cleaning :

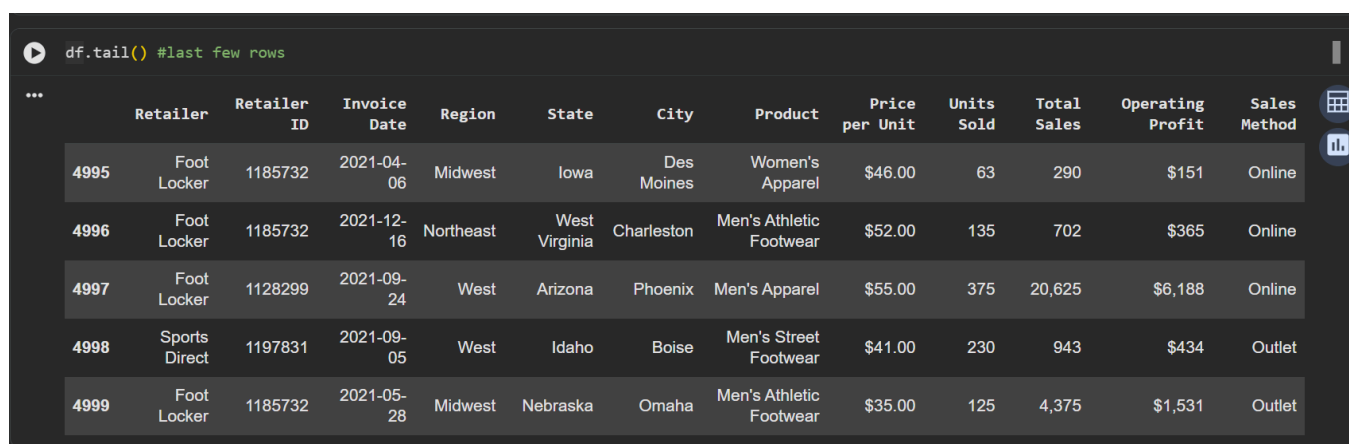
df.head(): Viewing the **first few rows** of the dataset helps get a quick understanding of the data structure and sample values before starting the detailed analysis.



```
df.head() # first few rows
```

	Retailer	Retailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Sales Method
0	Foot Locker	1128299	2021-06-24	Southeast	Virginia	Richmond	Men's Street Footwear	\$53.00	233	1,235	\$580	Online
1	Sports Direct	1197831	2021-08-18	South	Alabama	Birmingham	Women's Apparel	\$26.00	120	312	\$206	Outlet
2	Amazon	1185732	2021-06-11	Midwest	Ohio	Columbus	Women's Apparel	\$64.00	196	1,254	\$778	Online
3	Foot Locker	1128299	2021-10-23	West	Arizona	Phoenix	Women's Street Footwear	\$45.00	325	14,625	\$5,119	Online
4	West Gear	1128299	2021-01-31	West	California	San Francisco	Men's Athletic Footwear	\$60.00	875	52,500	\$7,875	Outlet

df.tail(): Viewing the **last few rows** helps confirm the dataset ends correctly and ensures there are no irregular or incomplete records at the bottom.



```
df.tail() #last few rows
```

```
...
```

	Retailer	Retailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Sales Method
4995	Foot Locker	1185732	2021-04-06	Midwest	Iowa	Des Moines	Women's Apparel	\$46.00	63	290	\$151	Online
4996	Foot Locker	1185732	2021-12-16	Northeast	West Virginia	Charleston	Men's Athletic Footwear	\$52.00	135	702	\$365	Online
4997	Foot Locker	1128299	2021-09-24	West	Arizona	Phoenix	Men's Apparel	\$55.00	375	20,625	\$6,188	Online
4998	Sports Direct	1197831	2021-09-05	West	Idaho	Boise	Men's Street Footwear	\$41.00	230	943	\$434	Outlet
4999	Foot Locker	1185732	2021-05-28	Midwest	Nebraska	Omaha	Men's Athletic Footwear	\$35.00	125	4,375	\$1,531	Outlet

df.shape: This shows the **number of rows and columns in the dataset**, giving a clear idea of the dataset size and structure.

```
df.shape #rows and columns  
  
(5000, 12)
```

df.dtypes: This displays the **data type of each column**, helping identify whether the values are stored in appropriate formats and which columns may require conversion during cleaning.

```
df.dtypes #data type of each column
```

Retailer	object
Retailer ID	int64
Invoice Date	object
Region	object
State	object
City	object
Product	object
Price per Unit	object
Units Sold	object
Total Sales	object
Operating Profit	object
Sales Method	object

dtype: object

df.info(): This provides an **overview of the dataset**, including column names, data types, non-null counts, and memory usage, helping assess the completeness and overall quality of the data.

```
[9] df.info() #basic info of the dataset  
✓ Os  
... <class 'pandas.core.frame.DataFrame'>  
RangeIndex: 5000 entries, 0 to 4999  
Data columns (total 12 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   Retailer              5000 non-null   object  
1   Retailer ID           5000 non-null   int64  
2   Invoice Date           5000 non-null   object  
3   Region                5000 non-null   object  
4   State                 5000 non-null   object  
5   City                  5000 non-null   object  
6   Product               5000 non-null   object  
7   Price per Unit         4998 non-null   object  
8   Units Sold            5000 non-null   object  
9   Total Sales           5000 non-null   object  
10  Operating Profit       5000 non-null   object  
11  Sales Method          5000 non-null   object  
dtypes: int64(1), object(11)  
memory usage: 468.9+ KB
```

df.describe(): This generates **statistical summaries for the numerical columns**, giving insights into the central tendency, spread, and distribution of key numerical variables.

```
df.describe() #summary statistics
```

	Retailer ID
count	5.000000e+03
mean	1.174212e+06
std	2.622155e+04
min	1.128299e+06
25%	1.185732e+06
50%	1.185732e+06
75%	1.185732e+06
max	1.197831e+06

df.index: This displays the **index range** of the dataset, helping confirm how the rows are numbered and ensuring there are no gaps or irregularities in the indexing.

```
df.index #the starting and ending index of the rows
RangeIndex(start=0, stop=5000, step=1)
```

df.isnull().sum(): This checks the **number of missing values in each column**, helping identify which fields may require cleaning or correction.

```
df.isnull().sum() #checking null values
```

Retailer	0
Retailer ID	0
Invoice Date	0
Region	0
State	0
City	0
Product	0
Price per Unit	2
Units Sold	0
Total Sales	0
Operating Profit	0
Sales Method	0

df.duplicated().sum(): This identifies **how many duplicate rows exist in the dataset**, helping determine whether any repeated records need to be removed during cleaning.

```
df.duplicated().sum()

np.int64(0)
```

df.nunique(): This shows the **number of unique values in each column**, helping understand the variety of data and identify categorical features with limited or diverse entries.

```
np.int64(0)

df.nunique() #unique
```

Retailer	6
Retailer ID	4
Invoice Date	673
Region	5
State	50
City	52
Product	6
Price per Unit	91
Units Sold	322
Total Sales	1405
Operating Profit	1250
Sales Method	3

dtype: int64

df.describe(include='object'): This provides a **summary of all categorical columns**, showing details like count, number of unique categories, most frequent value, and its frequency.

```
df.describe(include='object') #Summary of categorical columns
```

	Retailer	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Sales Method
count	5000	5000	5000	5000	5000	5000	4998	5000	5000	5000	5000
unique	6	673	5	50	52	6	91	322	1405	1250	3
top	Foot Locker	2021-03-16	West	Texas	Portland	Men's Street Footwear	\$50.00	225	10,000	\$6,300	Online
freq	1366	46	1246	232	166	848	352	108	37	23	2557

df[['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit']].head(): This previews key numeric-related columns **to check their values and ensure they are in the correct format** before starting the cleaning process.

```
df[['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit']].head()
# Preview of numeric-like columns before cleaning
```

	Price per Unit	Units Sold	Total Sales	Operating Profit
0	\$53.00	233	1,235	\$580
1	\$26.00	120	312	\$206
2	\$64.00	196	1,254	\$778
3	\$45.00	325	14,625	\$5,119
4	\$60.00	875	52,500	\$7,875

numerical_cols = df.columns[df.dtypes != 'object']: This identifies the **columns that are currently stored as numeric data types, helping separate them from categorical columns** before performing further cleaning.

```
numerical_cols = df.columns[df.dtypes != 'object'] # Identifying actual numeric columns (others will be cleaned in Stage 2)
print("Numerical Columns:")
numerical_cols

Numerical Columns:
Index(['Retailer ID'], dtype='object')
```

categorical_cols = df.columns[df.dtypes == 'object']: This identifies **all columns stored as object (text) types, including those that look numeric, helping determine which fields may need conversion** during cleaning.

```
categorical_cols = df.columns[df.dtypes == 'object'] # Identifying text-based columns (includes numeric-looking ones stored as object)
print("Categorical Columns:")
categorical_cols

Categorical Columns:
Index(['Retailer', 'Invoice Date', 'Region', 'State', 'City', 'Product',
      'Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit',
      'Sales Method'],
      dtype='object')
```

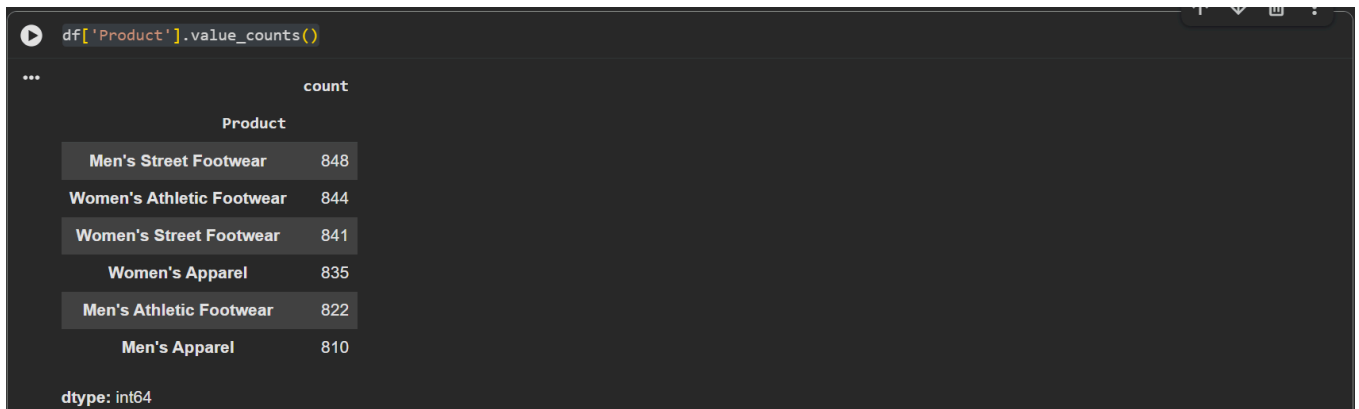
df['Region'].value_counts(): This shows **how many records belong to each region, helping understand the distribution of sales across different geographical areas.**

```
df['Region'].value_counts()
```

Region	count
West	1246
Northeast	1236
Midwest	973
South	931
Southeast	614

dtype: int64

df['Product'].value_counts(): This displays the **number of records for each product category**, helping identify which product types appear most frequently in the dataset.

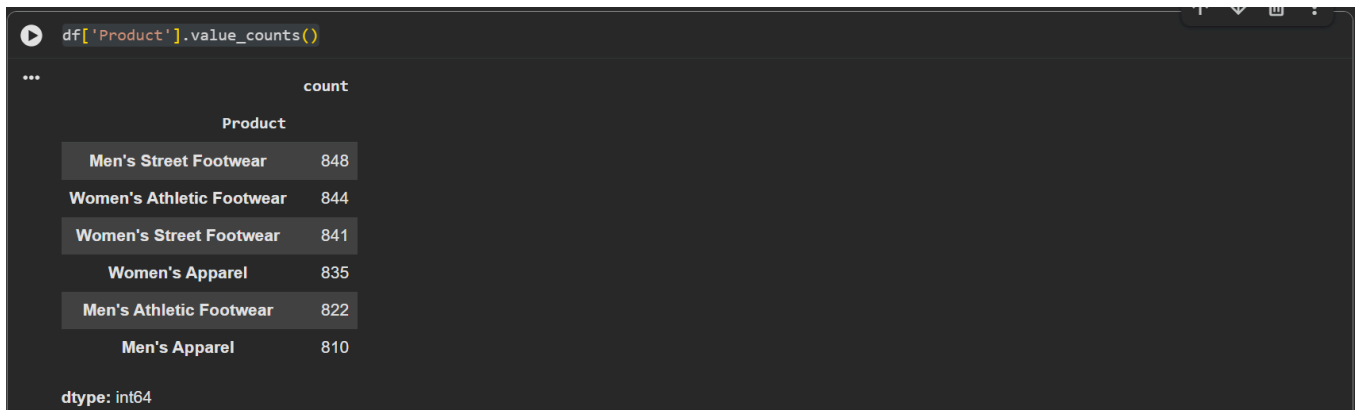


A Jupyter Notebook interface showing the execution of the code `df['Product'].value_counts()`. The output is a table with two columns: 'Product' and 'count'. The data is as follows:

Product	count
Men's Street Footwear	848
Women's Athletic Footwear	844
Women's Street Footwear	841
Women's Apparel	835
Men's Athletic Footwear	822
Men's Apparel	810

The dtype is int64.

df['Retailer'].value_counts(): This shows the **number of sales records for each retailer**, helping identify which retailers contribute most to the dataset.

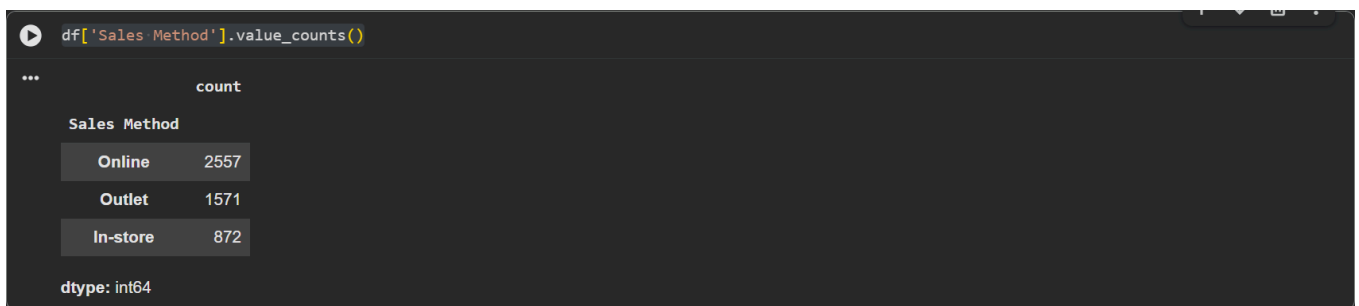


A Jupyter Notebook interface showing the execution of the code `df['Product'].value_counts()`. The output is a table with two columns: 'Product' and 'count'. The data is as follows:

Product	count
Men's Street Footwear	848
Women's Athletic Footwear	844
Women's Street Footwear	841
Women's Apparel	835
Men's Athletic Footwear	822
Men's Apparel	810

The dtype is int64.

df['Sales Method'].value_counts(): This shows **how many transactions occurred through each sales method (Online, In-store, Outlet)**, helping understand the distribution of sales channels.



A Jupyter Notebook interface showing the execution of the code `df['Sales Method'].value_counts()`. The output is a table with two columns: 'Sales Method' and 'count'. The data is as follows:

Sales Method	count
Online	2557
Outlet	1571
In-store	872

The dtype is int64.

Data Cleaning and Pre-processing:

Cleaning Numeric-like Columns by Removing Symbols and Converting to Float:

This step removes dollar signs and commas from the **numeric-like columns (Price per Unit, Units Sold, Total Sales, and Operating Profit)** and converts them into proper float format. This ensures that these values can be used accurately for calculations and visualisations. After cleaning, a quick preview confirms that the columns now contain clean numeric values.

```
# Cleaning numeric-like columns by removing $, commas and converting to float
# Clean and convert Price per Unit column
df['Price per Unit'] = df['Price per Unit'].astype(str).str.replace(r'[$,]', '', regex=True).astype(float)

# clean and convert 'Units Sold' column
df['Units Sold'] = df['Units Sold'].astype(str).str.replace(r'[$,]', '', regex=True).astype(float)

# clean and convert 'Total Sales' column
df['Total Sales'] = df['Total Sales'].astype(str).str.replace(r'[$,]', '', regex=True).astype(float)

# clean and convert 'Operating Profit' column
df['Operating Profit'] = df['Operating Profit'].astype(str).str.replace(r'[$,]', '', regex=True).astype(float)
```

```
# checking cleaned 'Price per Unit'
df['Price per Unit'].head()
```

Price per Unit

0	53.0
1	26.0
2	64.0
3	45.0
4	60.0

dtype: float64

```
# checking cleaned 'Units Sold'
df['Units Sold'].head()
```

Units Sold

0	233.0
1	120.0
2	196.0
3	325.0
4	875.0

dtype: float64

```
# checking cleaned 'Total Sales'
df['Total Sales'].head()
```

Total Sales

0	1235.0
1	312.0
2	1254.0
3	14625.0
4	52500.0

dtype: float64

```
# checking cleaned 'Operating Profit'
df['Operating Profit'].head()
```

Operating Profit

0	580.0
1	206.0
2	778.0
3	5119.0
4	7875.0

dtype: float64

Checking Datatypes After Cleaning:

This step verifies that the numeric-like columns have been successfully converted into the correct float data type after cleaning.

```
#Checking datatype of the columns after cleaning
df[['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit']].dtypes
```

	0
Price per Unit	float64
Units Sold	float64
Total Sales	float64
Operating Profit	float64

dtype: object

Handling Missing Values:

This step checks for missing values in each column after cleaning. The output shows that the **‘Price per Unit’ column contains 2 missing values**, while all other columns have no missing entries. These missing values will be handled in the next cleaning step to ensure the dataset remains complete and accurate.

```
#Handling missing values
#Checking missing values in each column after cleaning numeric columns
df.isnull().sum()
```

```
...
```

	0
Retailer	0
Retailer ID	0
Invoice Date	0
Region	0
State	0
City	0
Product	0
Price per Unit	2
Units Sold	0
Total Sales	0
Operating Profit	0

Identifying Rows with Missing 'Price per Unit' Values

This step displays the rows where the **'Price per Unit' column** contains **NaN values**. Reviewing these records helps understand which entries are incomplete and guides the decision on how to handle the missing values.

```
#Identifying rows where 'Price per Unit' is missing
df[df['Price per Unit'].isnull()]
```

	Retailer	Retailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Sales Method
2561	Foot Locker	1185732	2020-01-29	Northeast	New York	New York	Men's Apparel	NaN	850.0	51000.0	15300.0	Outlet
2787	Sports Direct	1197831	2020-07-29	South	Texas	Houston	Men's Apparel	NaN	500.0	20000.0	6000.0	Outlet

Handling Missing Values in 'Price per Unit'

The missing **'Price per Unit' values** were filled using the **median for the Men's Apparel category, which is 50**. The median was selected because it avoids the influence of extreme values and keeps the imputed price realistic. This imputation was performed within the Men's Apparel category only, rather than across the entire dataset, so the filled values match the typical pricing of that specific product group. This approach ensures the dataset remains complete without removing valid records, while maintaining consistent and reliable pricing information.

```
# Getting median price for Men's Apparel to fill missing values
median_mens_apparel = df.loc[df['Product'] == "Men's Apparel", 'Price per Unit'].median()
median_mens_apparel

50.0

# Imputing missing Price per Unit using Men's Apparel median
df.loc[df['Price per Unit'].isnull(), 'Price per Unit'] = median_mens_apparel

# Confirming that no missing values remain in Price per Unit
df['Price per Unit'].isnull().sum()

np.int64(0)
```

```
#checking particularly the missing values rows
df.loc[[2561, 2787], ['Product', 'Price per Unit']]

...
```

	Product	Price per Unit
2561	Men's Apparel	50.0
2787	Men's Apparel	50.0

Handling Duplicate Records

To ensure the dataset contains only unique sales entries, I checked for duplicate rows during the cleaning process. The initial check showed that there were no duplicate records (0 duplicates). I still applied `drop_duplicates()` as a precautionary step, and a second check confirmed that no duplicates were present. This ensures that the dataset remains clean, consistent, and free from repeated records.

```
# Checking for duplicate rows after cleaning
df.duplicated().sum()

np.int64(0)

# Removing duplicate rows if any
df = df.drop_duplicates()

# Checking again duplicates rows after dropping
df.duplicated().sum()

np.int64(0)
```

Summary Statistics for Numeric Columns:

This summary provides the basic statistical measures for the key numeric columns, helping to understand their overall distribution. The results show a wide range between minimum and maximum values, and some columns contain very high maximum values, suggesting the presence of potential outliers. These observations guide the next step of detailed outlier detection.

```
# Summary statistics for numeric columns to check ranges and possible extreme values
df[['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit']].describe()
```

...	Price per Unit	Units Sold	Total Sales	Operating Profit
count	5000.000000	5000.000000	5000.000000	5000.000000
mean	45.174200	253.424400	9131.546400	3379.28880
std	14.736665	211.243555	14126.242592	5407.82798
min	7.000000	0.000000	0.000000	0.000000
25%	35.000000	105.000000	422.750000	193.000000
50%	45.000000	176.000000	954.500000	436.000000
75%	55.000000	325.000000	14000.000000	5063.000000
max	110.000000	1250.000000	82500.000000	39000.000000

Outlier Detection Using the IQR Method:

The **Interquartile Range (IQR)** method was used to **detect outliers in the key numeric columns: Price per Unit, Units Sold, Total Sales, and Operating Profit. Outliers were identified but not removed, as these extreme values represent real business scenarios rather than errors.** In a sales dataset like this, unusually high prices, units sold, or profits often come from genuine events such as premium product purchases or bulk orders.

Column-wise Interpretation:

- **Price per Unit:** 43 outliers were detected above the typical pricing range. These likely represent premium-priced or discounted items and are valid in a retail setting.
- **Units Sold:** 331 outliers were found, mostly due to large-volume purchases. Bulk orders are normal in sales operations, so these values are meaningful.
- **Total Sales:** 385 outliers were identified, which typically occur when a high number of units are sold together, resulting in large transaction values.
- **Operating Profit:** 384 outliers were detected. These arise from high-profit or high-margin sales and reflect genuine business performance.

Conclusion: The outliers present in this dataset are not mistakes. They reflect genuine sales patterns such as large transactions, premium pricing, and high-profit orders. Since these extreme values provide important insights into real business behaviour, they were identified but intentionally not removed during the cleaning process.

```
# Detecting outliers in numeric columns using the IQR method (only checking, not removing)

numeric_cols = ['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit']

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)    # 25th percentile
    Q3 = df[col].quantile(0.75)    # 75th percentile
    IQR = Q3 - Q1                  # Interquartile Range

    lower_limit = Q1 - 1.5 * IQR    # Lower bound for outliers
    upper_limit = Q3 + 1.5 * IQR    # Upper bound for outliers

    outlier_count = ((df[col] < lower_limit) | (df[col] > upper_limit)).sum()

    print(f"Column: {col}")
    print(f"Outlier Lower Limit: {lower_limit}")
    print(f"Outlier Upper Limit: {upper_limit}")
    print(f"Number of outliers detected: {outlier_count}\n")
```

```
... Column: Price per Unit
Outlier Lower Limit: 5.0
Outlier Upper Limit: 85.0
Number of outliers detected: 43

Column: Units Sold
Outlier Lower Limit: -225.0
Outlier Upper Limit: 655.0
Number of outliers detected: 331

Column: Total Sales
Outlier Lower Limit: -19943.125
Outlier Upper Limit: 34365.875
Number of outliers detected: 385

Column: Operating Profit
Outlier Lower Limit: -7112.0
Outlier Upper Limit: 12368.0
Number of outliers detected: 384
```


Skewness Analysis of Numeric Columns:

Skewness was checked for all key numeric columns to understand the distribution of values. All four columns show right skewness, which is normal in sales datasets because a small number of large transactions or bulk orders tend to pull the distribution to the right. These patterns reflect real business behaviour, so no transformations were applied.

Column-wise Skewness Interpretation

- **Price per Unit (0.38):**
Slight positive skew and almost symmetric. This indicates regular pricing with occasional premium or discounted items.
- **Units Sold (1.51):**
Right-skewed due to occasional bulk purchases, which are normal in sales operations.
- **Total Sales (1.91):**
Right-skewed because some transactions involve very high sales amounts from large orders.
- **Operating Profit (2.30):**
Strong right skew, reflecting high profit events when many units are sold or when premium items are purchased.

Conclusion:

The skewness observed in the numeric columns is expected for a retail sales dataset. These values represent genuine business scenarios such as bulk orders and high-value transactions, so no transformation or correction was applied.

```
# Checking skewness for numeric columns
numeric_cols = ['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit']

print("Skewness of numeric columns:\n")
for col in numeric_cols:
    print(f"{col}: {df[col].skew():.2f}")

Skewness of numeric columns:

Price per Unit: 0.38
Units Sold: 1.51
Total Sales: 1.91
Operating Profit: 2.30
```

Converting Invoice Date & Extracting New Date Features:

The **'Invoice Date'** column was converted from string format to a proper datetime type to enable accurate time-based analysis. New date features—**Year, Month, and Quarter**—were extracted from this column to support trend analysis and visualisation across different time periods. The preview confirms that the new features were generated correctly.

```
# Converting 'Invoice Date' from string (object) to datetime format
df['Invoice Date'] = pd.to_datetime(df['Invoice Date'])

# Extracting new date-based features
df['Year'] = df['Invoice Date'].dt.year      # Extract year
df['Month'] = df['Invoice Date'].dt.month    # Extract month
df['Quarter'] = df['Invoice Date'].dt.quarter # Extract quarter

#checking the columns after extracting
df[['Invoice Date', 'Year', 'Month', 'Quarter']].head()
```

	Invoice Date	Year	Month	Quarter
0	2021-06-24	2021	6	2
1	2021-08-18	2021	8	3
2	2021-06-11	2021	6	2
3	2021-10-23	2021	10	4
4	2021-01-31	2021	1	1

Checking Datatypes of Extracted Columns:

This step verifies that the extracted date-based columns were created with the correct data types. The **'Invoice Date'** column is now stored in datetime format, while the newly created **Year, Month, and Quarter** columns are correctly stored as integer values. This ensures these fields can be used reliably in time-based analysis, grouping, and visualisations.

```
#checking the data type of the extracted columns
df[['Invoice Date', 'Year', 'Month', 'Quarter']].dtypes
```

	dtypes
Invoice Date	datetime64[ns]
Year	int32
Month	int32
Quarter	int32

dtype: object

Creating the Profit Margin (%) Column:

A new column, `Profit_Margin_%`, was created to measure how much profit is earned relative to total sales. It was calculated using the formula $(\text{Operating Profit} \div \text{Total Sales}) \times 100$. In cases where Total Sales is 0, the resulting value becomes NaN, **but these are not true missing values. Such cases correctly represent a no-sales, no-profit situation, so they were replaced with 0%.** The preview confirms that the new profit margin values are calculated correctly.

```
# Creating Profit Margin (%) column
df['Profit_Margin_%'] = (df['Operating Profit'] / df['Total Sales']) * 100

# Handling missing values in Profit_Margin_% (due to Total Sales = 0)
# These are not real missing values, so we replace them with 0%, which correctly represents a no-sales, no-profit scenario.
df['Profit_Margin_%'] = df['Profit_Margin_%'].fillna(0)

# Checking new column
df[['Operating Profit', 'Total Sales', 'Profit_Margin_%']].head()
```

	Operating Profit	Total Sales	Profit_Margin_%
0	580.0	1235.0	46.963563
1	206.0	312.0	66.025641
2	778.0	1254.0	62.041467
3	5119.0	14625.0	35.001709
4	7875.0	52500.0	15.000000

Validating the Total Sales Column:

To validate the accuracy of the ‘Total Sales’ column, a temporary column (`Calculated_Sales`) was created using the formula $\text{Price per Unit} \times \text{Units Sold}$. When comparing this calculated value with the actual Total Sales, more than 3,000 rows did not match. For example, in one row, the expected value based on 53×233 is 12,349, but the dataset shows Total Sales as 1,235. This clearly indicates that Total Sales in the dataset is already adjusted, most likely due to **retailer-level discounts, promotional pricing, bulk pricing, or other real-world deductions.**

Since **over 60% of the records follow this same pattern, it confirms that Total Sales represents the final adjusted transaction value and should not be recalculated or overwritten.** Creating a “Revenue per Unit” column would also lead to misleading results because the sales figures are discounted. Therefore, only Profit Margin (%) is used as the derived metric, and the temporary ‘Calculated_Sales’ column is removed after this validation.

```
# Checking relationship between Total Sales and (Price per Unit x Units Sold)
df['Calculated_Sales'] = df['Price per Unit'] * df['Units Sold']

# Comparing actual 'Total Sales' with calculated values for a few rows
df[['Price per Unit', 'Units Sold', 'Total Sales', 'Calculated_Sales']].head()
```

	Price per Unit	Units Sold	Total Sales	Calculated_Sales
0	53.0	233.0	1235.0	12349.0
1	26.0	120.0	312.0	3120.0
2	64.0	196.0	1254.0	12544.0
3	45.0	325.0	14625.0	14625.0
4	60.0	875.0	52500.0	52500.0

```
# Counting how many rows do NOT match Price x Units Sold
mismatch_count = (df['Total Sales'] != df['Calculated_Sales']).sum()
mismatch_count

np.int64(3015)
```

```
# Dropping Calculated_Sales (used only for validation)
df.drop(columns=['Calculated_Sales'], inplace=True)
```

EDA After Data Cleaning:

Dataset Shape:

This step checks the structure of the dataset after all cleaning and preprocessing steps. The **original dataset contained 5,000 rows and 12 columns. During preprocessing, four new columns were added — Year, Month, Quarter, and Profit_Margin_% — bringing the total to 16 columns.** The row count remains the same (5,000), confirming that no records were removed during cleaning and the full dataset has been preserved. This final shape verifies that the dataset is complete and ready for analysis and visualization.

```
# Number of rows and columns
rows = df.shape[0]
columns = df.shape[1]

print("Rows:", rows)
print("Columns:", columns)

Rows: 5000
Columns: 16
```

Datatype Validation After Cleaning:

A datatype check was performed to confirm that all columns were correctly formatted after the cleaning and preprocessing steps. Initially, several numeric fields such as **Price per Unit, Units Sold, Total Sales, and Operating Profit** were stored as **object types** because they contained symbols like commas and dollar signs. After cleaning, all these columns have been successfully **converted to float format**. The **Invoice Date column**, which originally appeared as a **string**, is now correctly converted to **datetime64** format.

Additionally, **new date-based features (Year, Month, Quarter) extracted during preprocessing are stored as integer types, and the newly created Profit_Margin_% column is stored as float64.** Categorical columns such as Retailer, Region, State, City, Product, and Sales Method remain in object format. This confirms that the dataset now has accurate and appropriate datatypes for analysis and visualization.

```
# Data types of each column
print("\nData Types:")
print(df.dtypes)
```

```

...
Data Types:
Retailer          object
Retailer ID       int64
Invoice Date      datetime64[ns]
Region            object
State             object
City              object
Product           object
Price per Unit    float64
Units Sold        float64
Total Sales       float64
Operating Profit   float64
Sales Method      object
Year              int32
Month             int32
Quarter           int32
Profit_Margin_%   float64
dtype: object

```

Missing Values After Cleaning:

A final missing value check was performed to ensure that all null entries were properly handled during the cleaning process. In the raw dataset, the **‘Price per Unit’** column contained **missing values**, which were **imputed using the median price specific to the Men’s Apparel category**. After this correction and the full cleaning workflow, the dataset shows **zero missing values across all 16 columns**. This confirms that the dataset is now complete, consistent, and ready for further analysis and visualization.

```

▶ # Checking missing values after cleaning
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())

```

```

...
Missing Values After Cleaning:
Retailer          0
Retailer ID       0
Invoice Date      0
Region            0
State             0
City              0
Product           0
Price per Unit    0
Units Sold        0
Total Sales       0
Operating Profit   0
Sales Method      0
Year              0
Month             0
Quarter           0
Profit_Margin_%   0
dtype: int64

```

Preview of the Cleaned Dataset:

A preview of the first few rows of the cleaned dataset confirms that all preprocessing steps were applied successfully. **Numeric columns such as Price per Unit, Units Sold, Total Sales, and Operating Profit are now in clean float format. The Invoice Date column appears in proper datetime format, and the newly created date-based features — Year, Month, and Quarter — are correctly populated. The derived Profit_Margin_% column is also visible and accurately reflects the profit percentage for each transaction.** This preview verifies that the dataset is well-organized, consistent, and ready for visualization and further analysis.

```
# First few rows
df.head()
```

stailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Sales Method	Year	Month	Quarter	Profit_Margin_%
1128299	2021-06-24	Southeast	Virginia	Richmond	Men's Street Footwear	53.0	233.0	1235.0	580.0	Online	2021	6	2	46.963563
1197831	2021-08-18	South	Alabama	Birmingham	Women's Apparel	26.0	120.0	312.0	206.0	Outlet	2021	8	3	66.025641
1185732	2021-06-11	Midwest	Ohio	Columbus	Women's Apparel	64.0	196.0	1254.0	778.0	Online	2021	6	2	62.041467
1128299	2021-10-23	West	Arizona	Phoenix	Women's Street Footwear	45.0	325.0	14625.0	5119.0	Online	2021	10	4	35.001709
1128299	2021-01-31	West	California	San Francisco	Men's Athletic Footwear	60.0	875.0	52500.0	7875.0	Outlet	2021	1	1	15.000000

Data Visualisation:

Chart 1: Region Distribution (Count Plot):

Code:

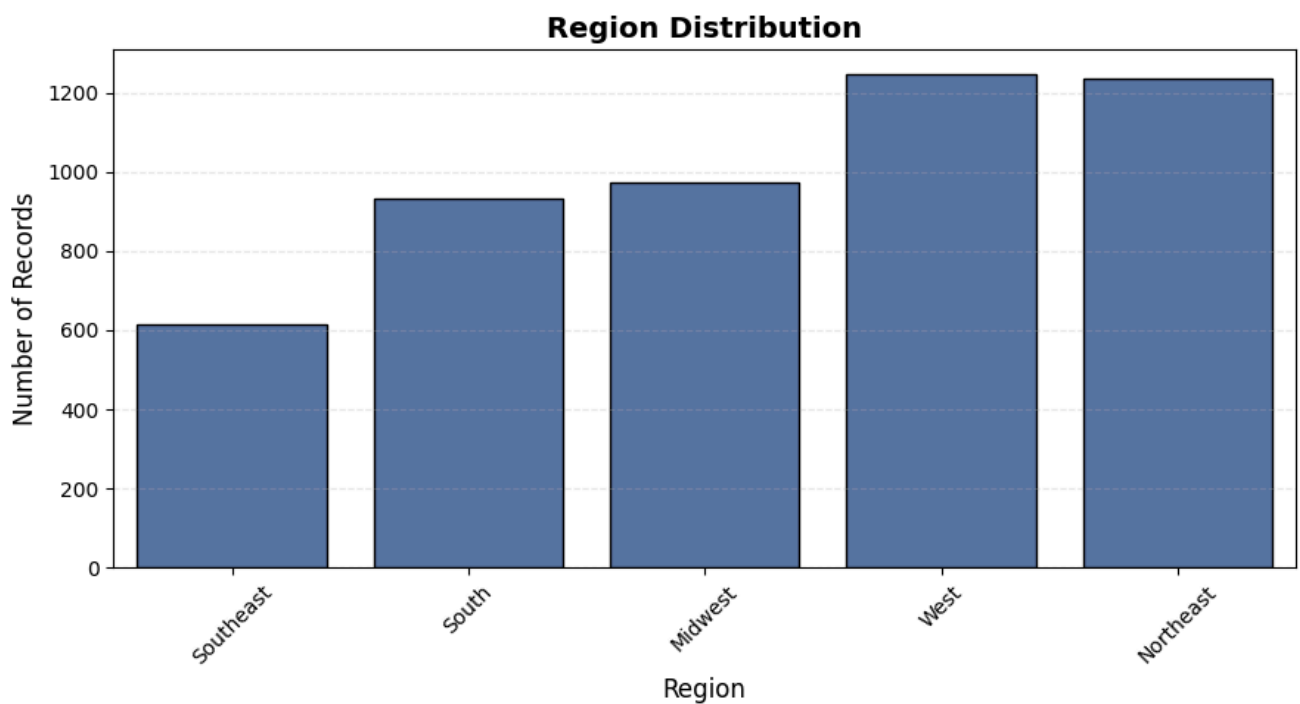
```
#Chart 1 - Region Distribution (Countplot)

plt.figure(figsize=(9,5))

sns.countplot(
    data=df,
    x='Region',
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Region Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Region", fontsize=12)
plt.ylabel("Number of Records", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This chart displays the **number of sales records from each region**, giving a clear view of how the dataset is geographically distributed. It shows how many transactions were recorded across the West, Northeast, Midwest, South, and Southeast regions.

Features Used

- **Region** (categorical feature)
- Count of transactions for each region

Insight / Interpretation

The **West and Northeast regions have the highest number of sales entries**, indicating stronger sales activity in these areas. **The Midwest and South show moderate transaction levels**, while the **Southeast has the lowest number of records**. This suggests that Adidas has stronger market presence or higher sales activity in the western and northeastern parts of the United States compared to the Southeast region.

Chart 2: Product Category Distribution (Count Plot) :

Code:

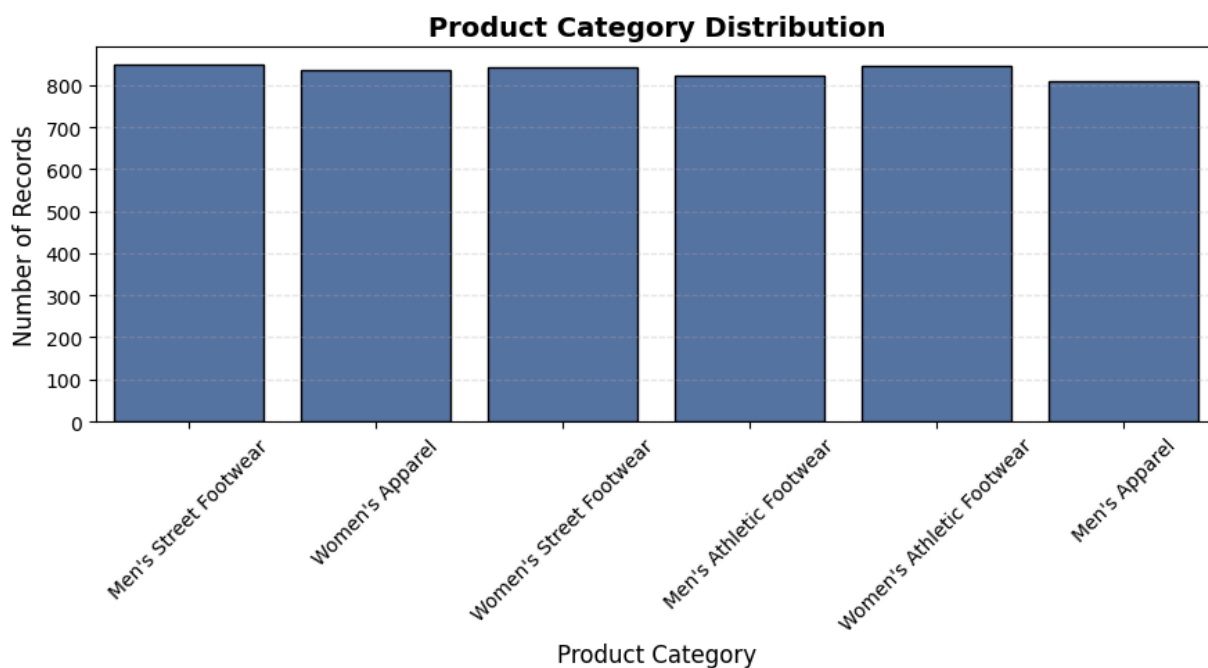
```
#Chart 2 - Product Distribution (Countplot)

plt.figure(figsize=(9,5))

sns.countplot(
    data=df,
    x='Product',
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Product Category Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Product Category", fontsize=12)
plt.ylabel("Number of Records", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This chart shows the **number of sales records for each Adidas product category**. It provides a clear view of how different product types—such as men’s and women’s footwear and apparel—are represented in the dataset.

Features Used

- **Product** (categorical feature)
- Count of transactions for each product category

Insight / Interpretation

The distribution across product categories is fairly balanced, but **men’s and women’s footwear categories have slightly higher transaction counts compared to apparel**. This suggests that **footwear products—both street and athletic—appear more frequently in sales records and may have stronger customer demand**. Apparel categories also show significant activity but are mildly lower, indicating **footwear is the more dominant product segment** in this dataset.

Chart 3: Retailer Distribution (Count Plot):

Code:

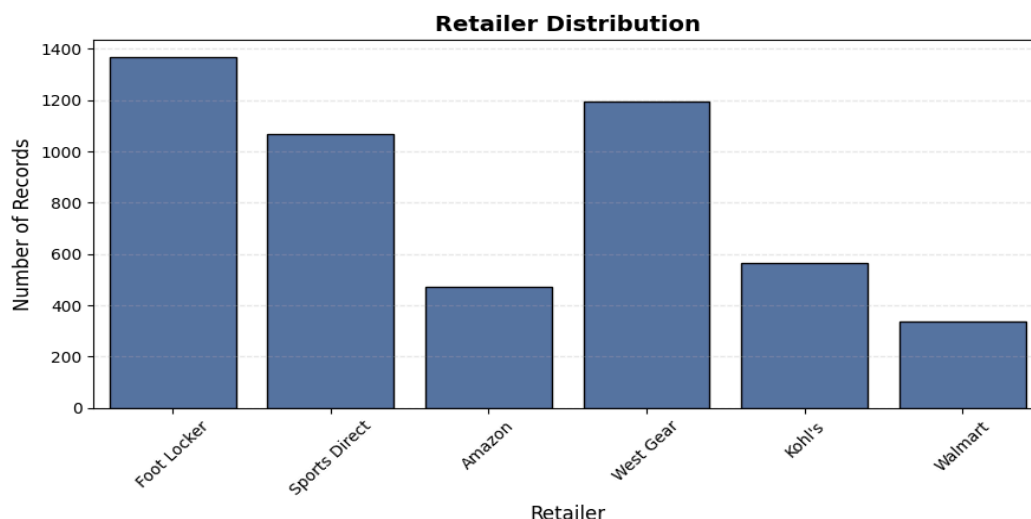
```
#Chart 3 - Retailer Distribution (Countplot)

plt.figure(figsize=(9,5))

sns.countplot(
    data=df,
    x='Retailer',
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Retailer Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Retailer", fontsize=12)
plt.ylabel("Number of Records", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This chart shows the **number of sales records associated with each retailer**. It provides an overview of how frequently Adidas products were sold across different retail partners in the dataset.

Features Used

- **Retailer** (categorical feature)
- Count of transactions per retailer

Insight / Interpretation

Foot Locker, West Gear, and Sports Direct show the highest number of recorded transactions, indicating that they are major sales channels for Adidas in this dataset. **Amazon, Kohl's, and Walmart have noticeably fewer entries, suggesting lower transaction volume** or less representation in this particular dataset. Overall, the chart highlights strong retailer-wise variation, with certain partners contributing significantly more to Adidas sales activity than others.

Chart 4: Sales Method Distribution (Count Plot):

Code:

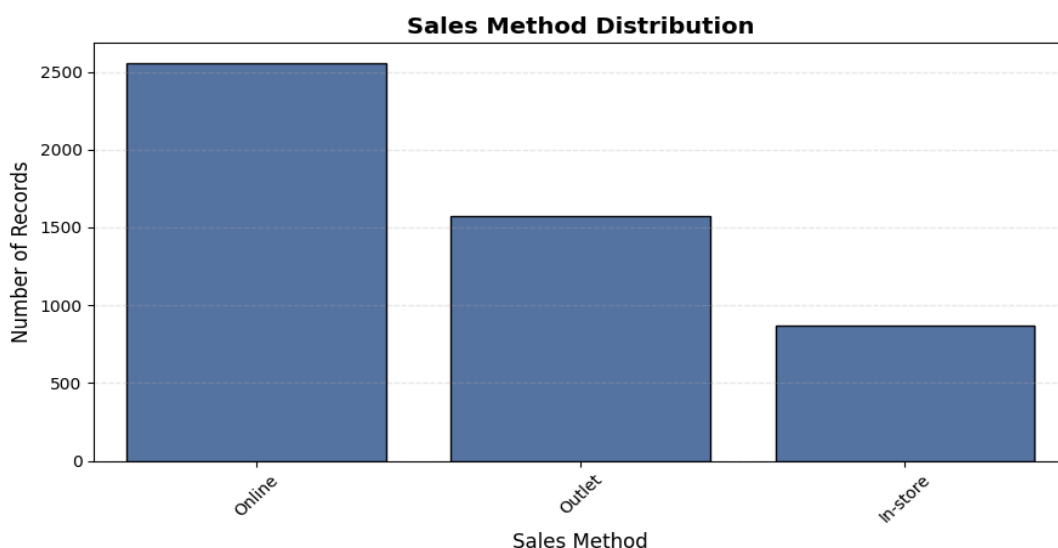
```
#Chart 4 - Sales Method Distribution (Countplot)

plt.figure(figsize=(9,5))

sns.countplot(
    data=df,
    x='Sales Method',
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Sales Method Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Sales Method", fontsize=12)
plt.ylabel("Number of Records", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This chart shows the **number of sales transactions completed through each sales method — Online, Outlet, and In-store**. It provides a clear view of how customers choose to purchase Adidas products within this dataset.

Features Used

- **Sales Method** (categorical feature)
- Count of transactions for each method

Insight / Interpretation

Online sales represent the largest share of transactions, indicating that most customers prefer purchasing Adidas products digitally. Outlet sales are the second-highest, showing continued usage of factory outlet stores. In-store sales have the lowest count, suggesting that fewer customers are buying directly from physical retail stores compared to online and outlet channels. **Overall, the chart highlights a strong shift toward online purchasing behaviour, reflecting modern shopping trends.**

Chart 5: Distribution of Units Sold (Histogram):

Code:

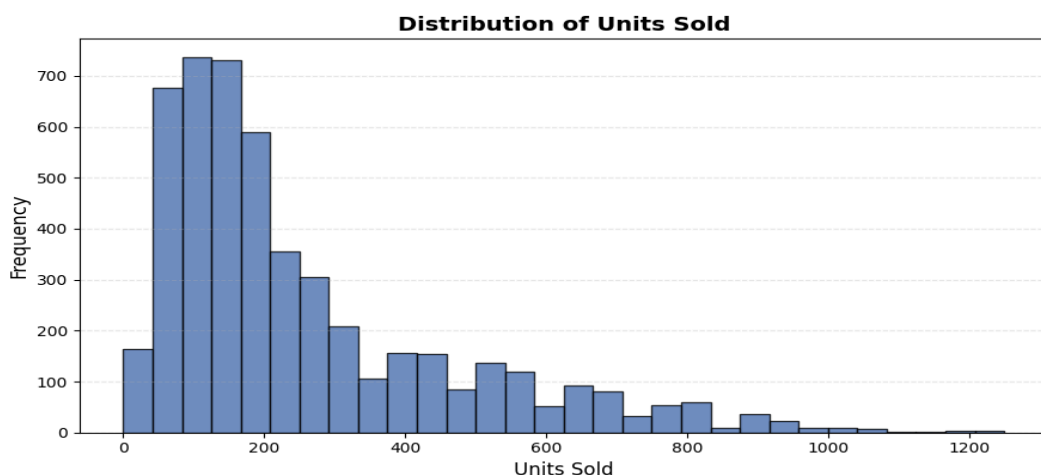
```
#Chart 5 - Distribution of Units Sold (Histogram)

plt.figure(figsize=(9,5))

plt.hist(
    df['Units Sold'],
    bins=30,
    color='#4C72B0',
    edgecolor='black',
    alpha=0.8
)

plt.title("Distribution of Units Sold", fontsize=14, fontweight='bold')
plt.xlabel("Units Sold", fontsize=12)
plt.ylabel("Frequency", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This histogram illustrates **how the number of units sold is distributed across all transactions** in the dataset. It shows **how often different quantities of Adidas products were purchased**.

Features Used

- **Units Sold** (numerical feature)
- Frequency distribution based on bin intervals

Insight / Interpretation

The distribution is strongly **right-skewed**, meaning **most transactions involve relatively small quantities**. **Only a small number of orders contain very high unit counts, representing bulk purchases**. This pattern is typical in retail sales, where **most customers buy one or few items, and larger orders are less common**. The chart clearly highlights that day-to-day sales are driven primarily by low-quantity purchases, while high-volume orders appear infrequently.

Chart 6: Boxplot of Total Sales:

Code:

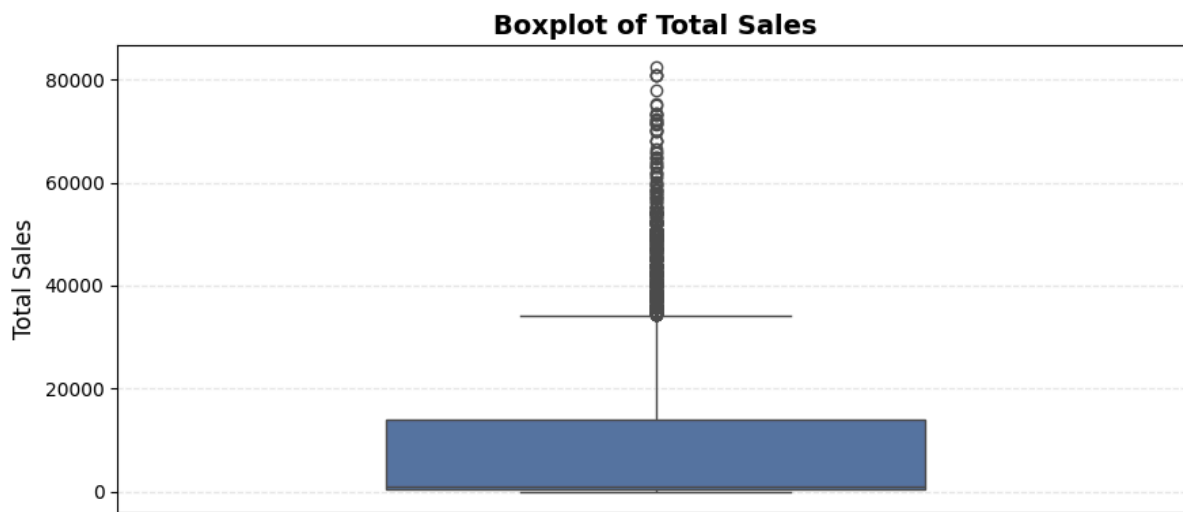
```
#Chart 6 - Total Sales Boxplot (Outlier Detection)

plt.figure(figsize=(9,4))

sns.boxplot(
    data=df,
    y='Total Sales',
    color='#4C72B0',
    width=0.5
)

plt.title("Boxplot of Total Sales", fontsize=14, fontweight='bold')
plt.ylabel("Total Sales", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This boxplot shows **how Total Sales values are distributed across all transactions. It highlights the central range of sales amounts and identifies unusually large transactions that appear as outliers.**

Features Used

- **Total Sales** (numerical feature)
- Boxplot visualization to show spread, median, and outliers

Insight / Interpretation

Most sales amounts fall within a moderate range, but the boxplot shows many high-value outliers above the upper whisker. These outliers represent genuinely large or exceptional sales transactions, often resulting from bulk purchases, premium products, or retailer-level pricing adjustments. Their presence is typical in retail sales data and does not indicate errors. Instead, it reflects natural variability in sales performance and shows that high-value orders occur frequently enough to influence overall revenue trends.

Chart 7: Distribution of Total Sales (Histogram):

Code:

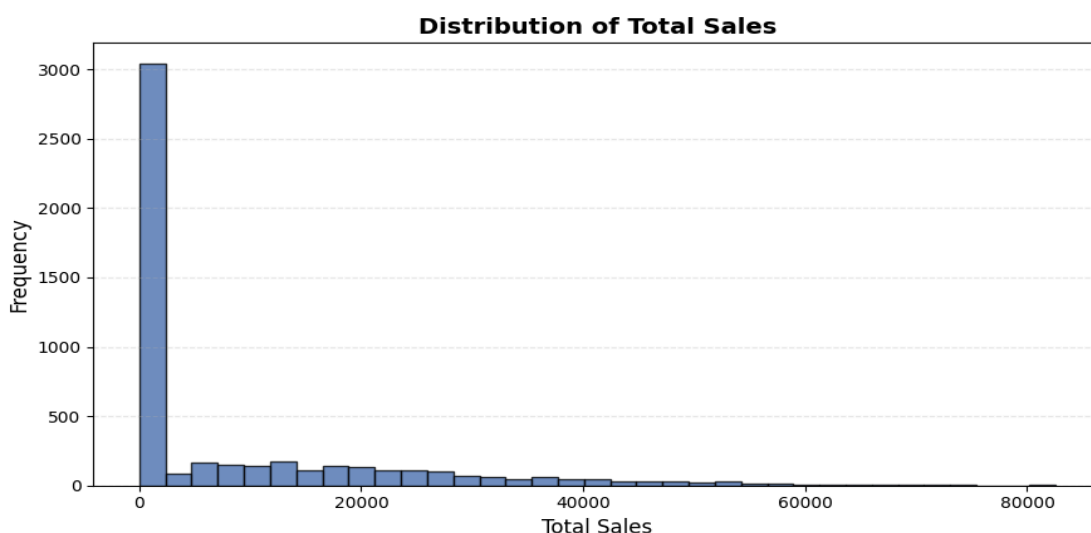
```
#Chart 7 - Distribution of Total Sales (Histogram)

plt.figure(figsize=(9,5))

plt.hist(
    df['Total Sales'],
    bins=35,
    color='#4C72B0',
    edgecolor='black',
    alpha=0.8
)

plt.title("Distribution of Total Sales", fontsize=14, fontweight='bold')
plt.xlabel("Total Sales", fontsize=12)
plt.ylabel("Frequency", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This histogram illustrates the **distribution of Total Sales across all transactions**. It shows how frequently different sales amounts occur, **helping to understand the overall spread of revenue** in the dataset.

Features Used

- **Total Sales** (numerical feature)
- Frequency distribution using histogram bins

Insight / Interpretation

The distribution is strongly **right-skewed**. **Most transactions fall within the lower sales range, indicating that typical purchases tend to have smaller total values. Only a small number of transactions appear on the higher end of the sales scale, representing large-value or bulk orders.** This pattern is common in retail, where regular purchases happen frequently, while high-value transactions are less common but still meaningful. The chart confirms that **smaller sales dominate the dataset, while large transactions occur occasionally and contribute to revenue spikes.**

Chart 8: Correlation Heatmap of Numerical Variables:

Code:

```
#Chart 8 - Correlation Heatmap of Numerical Features

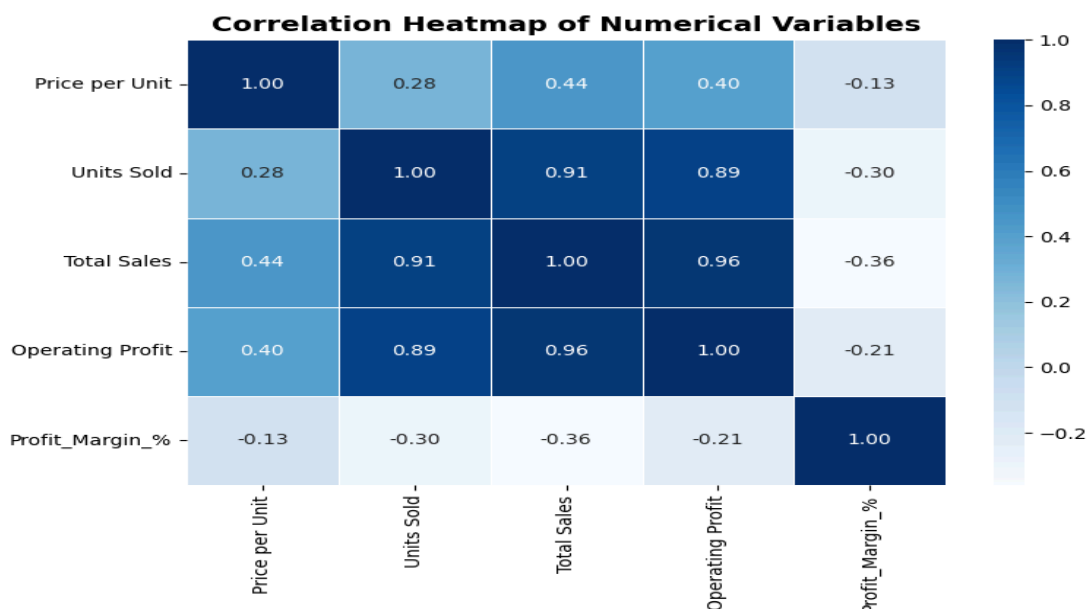
plt.figure(figsize=(8,6))

corr_matrix = df[['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit', 'Profit_Margin_%']].corr()

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='Blues',
    fmt=".2f",
    linewidths=0.5
)

plt.title("Correlation Heatmap of Numerical Variables", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Output:



Description

This heatmap displays the correlation strength between the key numerical variables in the dataset. The color intensity and values help identify how closely different sales and profit metrics move together.

Features Used

Numerical columns: Price per Unit, Units Sold, Total Sales, Operating Profit, Profit_Margin_%.

Insight / Interpretation

Total Sales shows a very strong positive correlation with both Units Sold and Operating Profit, which is expected since selling more units increases revenue and typically raises profit. Operating Profit also has a high correlation with Units Sold, reinforcing that higher volume drives profitability.

In contrast, Profit_Margin_% has weak correlations with most variables, indicating that profit percentage is influenced by factors beyond just sales volume or pricing — such as discounts, cost variations, or retailer-level adjustments. Overall, the heatmap highlights which variables strongly drive sales performance and which metrics behave independently.

Chart 9: Total Sales by Region (Bar chart)

Code:

```
#Chart 9 - Total Sales by Region (Bar Chart)

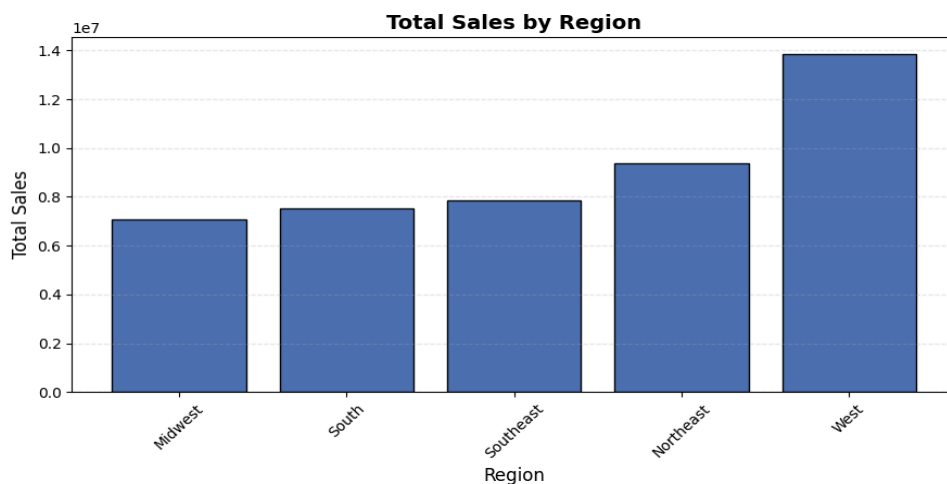
region_sales = df.groupby('Region')['Total Sales'].sum().sort_values()

plt.figure(figsize=(9,5))

plt.bar(
    region_sales.index,
    region_sales.values,
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Total Sales by Region", fontsize=14, fontweight='bold')
plt.xlabel("Region", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This bar chart shows the **total sales generated from each region by summing all transactions** within that area. It provides a clear comparison of revenue contribution across different geographic markets.

Features Used

- **Region** (categorical feature)
- **Total Sales** (numerical feature, aggregated using sum)
- Grouped by region to compare total revenue

Insight / Interpretation

The **West region generates the highest total sales**, making it the strongest-performing market for Adidas in this dataset. The **Northeast region also shows strong revenue performance**. In contrast, the Midwest region has the lowest total sales, indicating weaker market contribution. This variation highlights that **certain regions are significantly more profitable and play a larger role in overall revenue**, while others may require targeted strategies to boost performance.

Chart 10: Total Sales by Product Category

Code:

```
#Chart 10 - Total Sales by Product (Bar Chart)

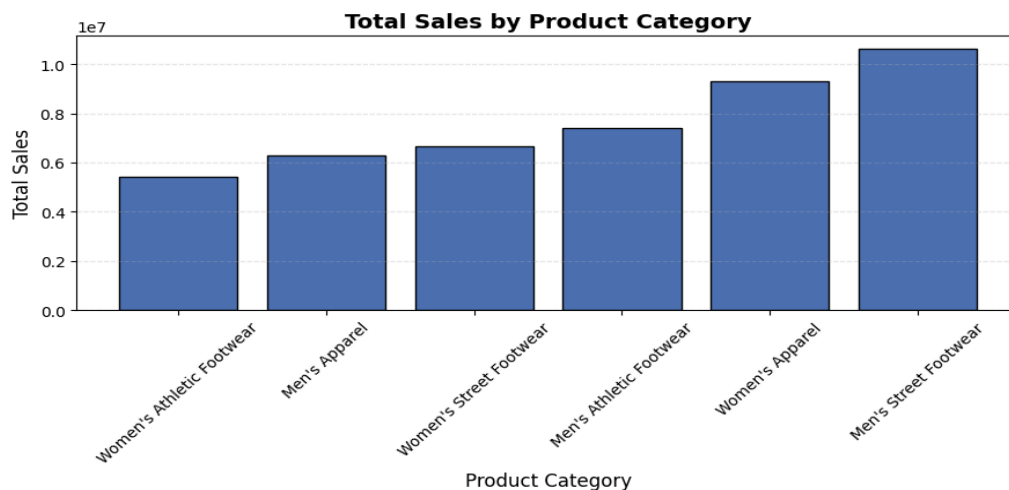
product_sales = df.groupby('Product')['Total Sales'].sum().sort_values()

plt.figure(figsize=(9,5))

plt.bar(
    product_sales.index,
    product_sales.values,
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Total Sales by Product Category", fontsize=14, fontweight='bold')
plt.xlabel("Product Category", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This bar chart shows the **total revenue generated by each product category**. By summing the Total Sales values for every product type, the chart **compares how different categories contribute to Adidas' overall sales performance**.

Features Used

- **Product** (categorical feature)
- **Total Sales** (numerical feature, aggregated using sum)
- Grouped by product category

Insight / Interpretation

Footwear categories—both Men's and Women's—generate the highest total sales, making them the strongest revenue drivers in this dataset. Apparel categories for both men and women contribute notably less, indicating lower purchase value or buying frequency. Overall, the chart shows that **Adidas earns significantly more from footwear products than apparel, highlighting footwear as the leading product segment** in terms of revenue.

Chart 11: Average Units Sold by Sales Method

Code:

```
#Chart 11 - Average Units Sold by Sales Method (Bar Chart)

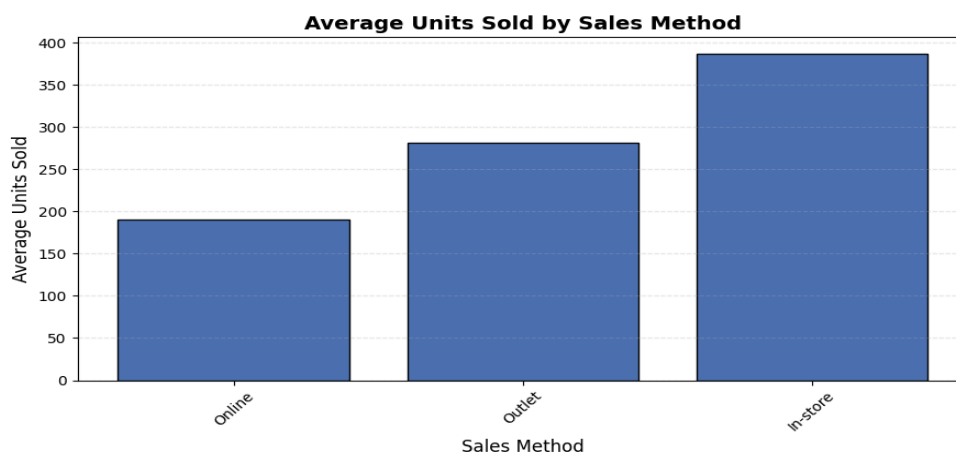
avg_units_sales_method = df.groupby('Sales Method')['Units Sold'].mean().sort_values()

plt.figure(figsize=(9,5))

plt.bar(
    avg_units_sales_method.index,
    avg_units_sales_method.values,
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Average Units Sold by Sales Method", fontsize=14, fontweight='bold')
plt.xlabel("Sales Method", fontsize=12)
plt.ylabel("Average Units Sold", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This bar chart compares the **average number of units sold per transaction** across three sales channels: **Online**, **Outlet**, and **In-store**. It shows how customer buying quantity varies depending on where they make the purchase.

Features Used

- **Sales Method** (categorical feature)
- **Units Sold** (numerical feature aggregated using mean)
- Grouped by sales method

Insight / Interpretation

The chart shows that **In-store purchases have the highest average units sold**, indicating that customers tend to buy more items when visiting physical stores. This could be driven by in-store promotions, sales assistance, or customers browsing multiple items.

Outlet stores have a moderate average quantity, suggesting buyers still purchase multiple items but not as much as in physical retail stores.

Online purchases show the lowest average units sold. This is a typical pattern in e-commerce where customers usually buy one or two specific items rather than making bulk purchases.

What This Reveals

- Physical stores encourage **multi-item or bulk purchases**.
- Online transactions tend to be **single-item focused**.
- Outlet stores fall in between, showing moderately high engagement.

This insight helps Adidas understand **which sales channels drive higher quantity purchases**, supporting decisions related to channel-wise marketing, inventory planning, and promotional strategy.

Chart 12: Operating Profit by Retailer

Code:

```
#Chart 12 - Operating Profit by Retailer (Bar Chart)

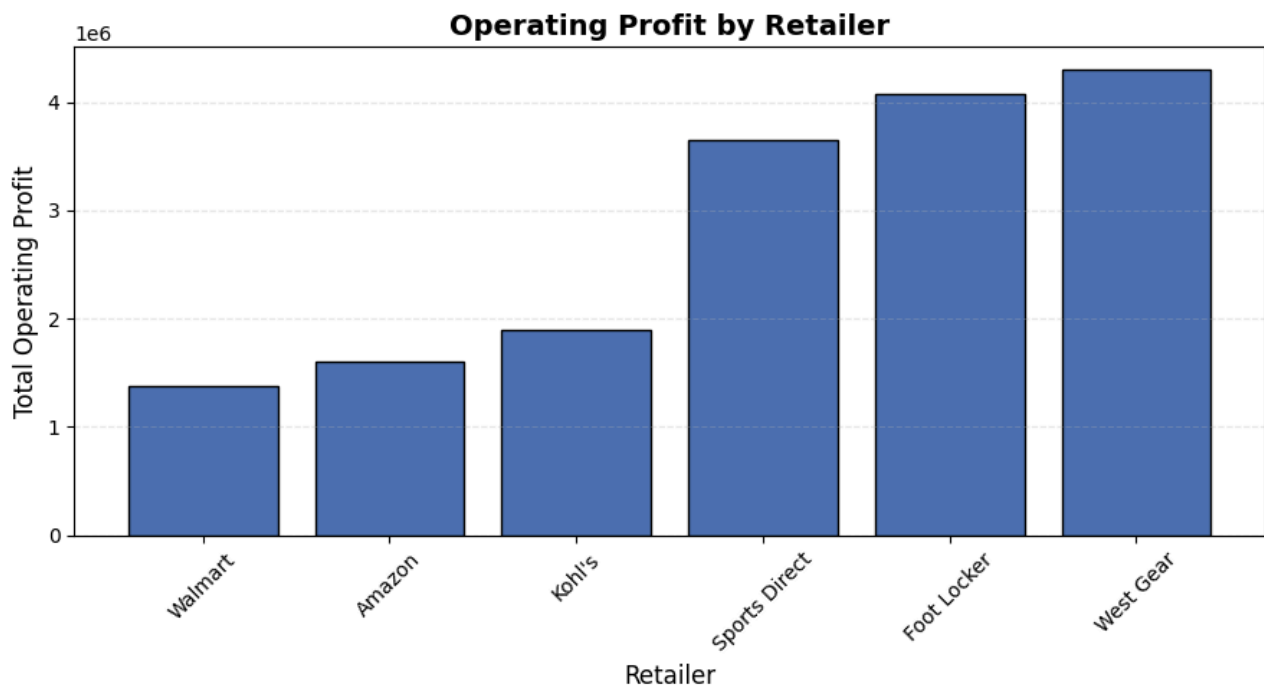
retailer_profit = df.groupby('Retailer')['Operating Profit'].sum().sort_values()

plt.figure(figsize=(9,5))

plt.bar(
    retailer_profit.index,
    retailer_profit.values,
    color='#4C72B0',
    edgecolor='black'
)

plt.title("Operating Profit by Retailer", fontsize=14, fontweight='bold')
plt.xlabel("Retailer", fontsize=12)
plt.ylabel("Total Operating Profit", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This bar chart compares the **total operating profit** generated by each retailer. By summing the Operating Profit values for each retailer, the chart highlights which partners contribute most to Adidas' overall profitability.

Features Used

- **Retailer** (categorical feature)
- **Operating Profit** (numerical feature aggregated using sum)
- Grouped by retailer

Insight / Interpretation

The chart shows a clear difference in profit contribution across retailers:

- **West Gear** and **Foot Locker** generate the highest total operating profit, making them Adidas' strongest profit drivers in this dataset.
- **Sports Direct** also performs well, contributing significantly to overall profitability.
- **Kohl's**, **Amazon**, and **Walmart** show comparatively lower profit totals.

It is important to note that **higher profit does not always mean more sales entries**.

A retailer can appear lower in transaction count but still have higher operating profit if:

- They sell higher-margin products
- They have fewer discounts or promotions
- They sell premium items
- Their profit per transaction is higher

This explains why **West Gear**, despite having fewer entries than some retailers, still appears as the top profit contributor.

What This Chart Shows

- Which retailers generate the **most and least profit**
- The impact of retailer-wise pricing strategies and margins
- Key retail partners who significantly drive financial performance

This insight helps Adidas understand which retailers bring higher returns and where business decisions such as partnerships, marketing efforts, and stock allocation can be optimized.

Chart 13: Units Sold vs Total Sales (Scatter Plot)

Code:

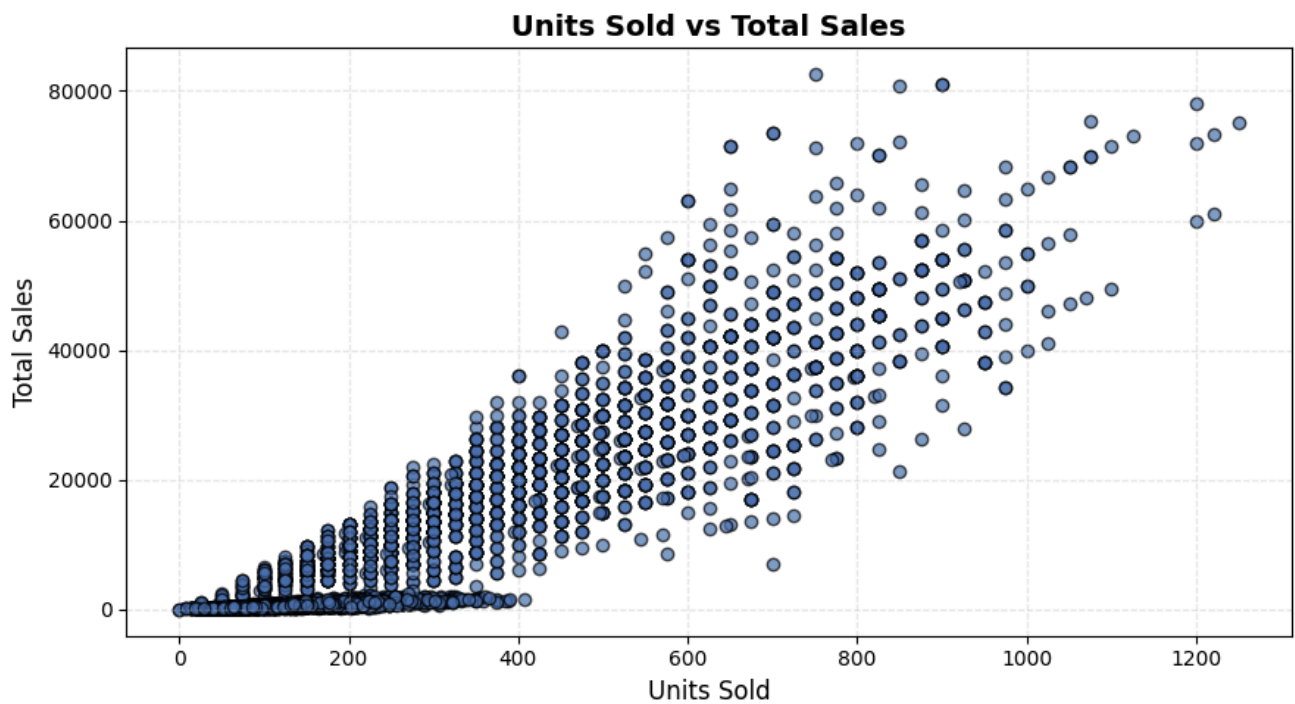
```
#Chart 13 - Units Sold vs Total Sales (Scatterplot)

plt.figure(figsize=(9,5))

plt.scatter(
    df['Units Sold'],
    df['Total Sales'],
    color='#4C72B0',
    edgecolor='black',
    alpha=0.7
)

plt.title("Units Sold vs Total Sales", fontsize=14, fontweight='bold')
plt.xlabel("Units Sold", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.grid(linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This scatter plot visualizes the relationship between **Units Sold** and **Total Sales**. Each point represents a single sales transaction, showing how quantity purchased relates to revenue generated.

Features Used

- **Units Sold** (numerical variable)
 - **Total Sales** (numerical variable)
- These two features were plotted against each other to study how they move together.

Insight / Interpretation

The plot shows a **strong positive relationship** between Units Sold and Total Sales:

- As **Units Sold increases**, the **Total Sales** also increases.
- Transactions with **very high quantities** appear as points placed much higher on the graph, representing **bulk orders** or **large-value purchases**.
- The upward pattern confirms that selling more units naturally leads to higher revenue, which is expected in retail datasets.

The presence of scattered high-value points indicates occasional large purchases, which significantly boost revenue even though they occur less frequently.

What This Chart Shows

- How strongly revenue depends on the quantity sold
- The presence of bulk or high-value transactions
- A clear upward trend showing that **Units Sold is a key driver of Total Sales**
- Validation of normal business behavior — more units purchased → higher sales

This scatter plot helps illustrate the core relationship between quantity and revenue, reinforcing the importance of high-volume transactions in overall sales performance.

Chart 14: Total Sales vs Operating Profit (Scatter Plot)

Code:

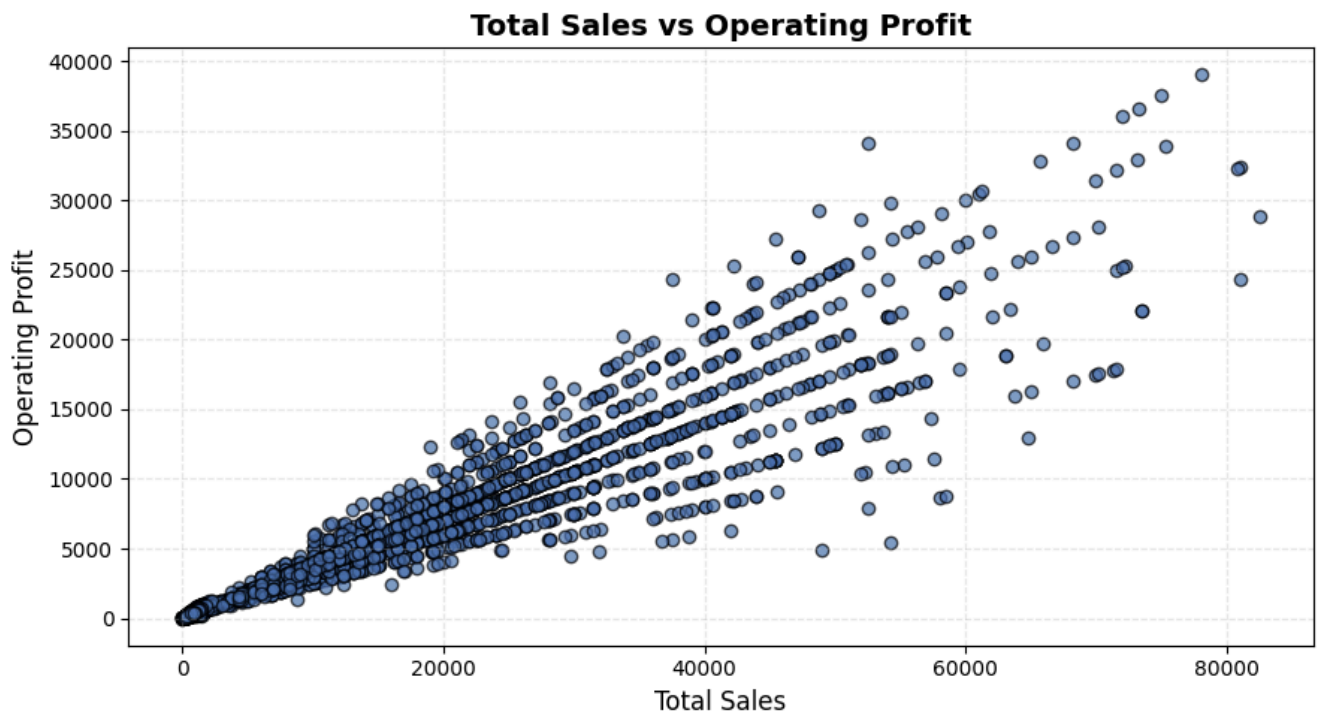
```
#Chart 14 - Total Sales vs Operating Profit (Scatterplot)

plt.figure(figsize=(9,5))

plt.scatter(
    df['Total Sales'],
    df['Operating Profit'],
    color='#4C72B0',
    edgecolor='black',
    alpha=0.7
)

plt.title("Total Sales vs Operating Profit", fontsize=14, fontweight='bold')
plt.xlabel("Total Sales", fontsize=12)
plt.ylabel("Operating Profit", fontsize=12)
plt.grid(linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This scatter plot illustrates the relationship between **Total Sales** and **Operating Profit** for each transaction. Every dot represents one sale, showing how much revenue was generated and how much profit was earned from that specific transaction.

Features Used

- **Total Sales** (numerical variable)
- **Operating Profit** (numerical variable)

Plotting these two metrics together helps identify how changes in sales impact profitability.

Insight / Interpretation

The plot shows a **strong positive relationship** between Total Sales and Operating Profit:

- As **Total Sales increases**, **Operating Profit also increases**.
- The upward trend confirms that higher-value transactions generally result in higher profit.
- Several points appear significantly above the main cluster — these represent **exceptionally profitable transactions**, often due to:
 - large quantities sold
 - premium or high-margin products
 - retailer-specific pricing or promotional effects

The scatter pattern clearly demonstrates that profit growth is closely tied to revenue growth.

What This Chart Shows

- Profit rises consistently with higher sales amounts.
- High-profit transactions are almost always linked to high-revenue transactions.
- The strong positive correlation indicates that **revenue is a major driver of operating profit**.
- A few very high-profit points highlight the importance of large or premium orders.

This chart reinforces the business insight that **increasing total sales directly boosts profitability**, making it a key performance relationship in the dataset.

Chart 15: Monthly Sales Trend (Line Chart)

Code:

```
#Chart 15 - Monthly Sales Trend (Line Chart)

monthly_sales = df.groupby(['Year', 'Month'])['Total Sales'].sum()

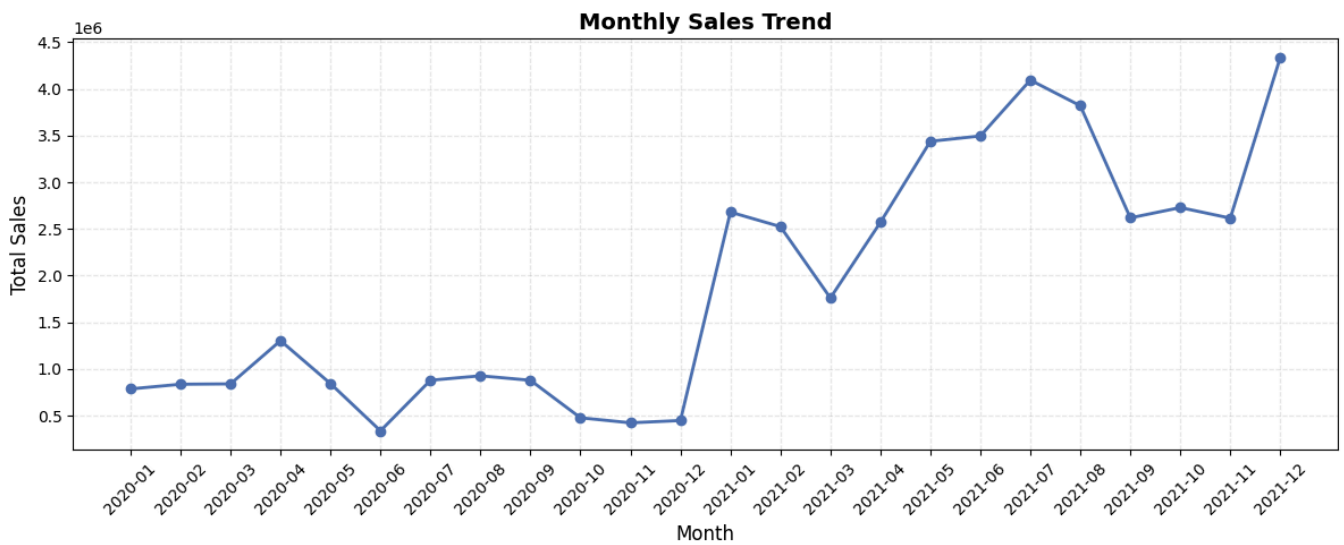
# Creating a proper monthly index for plotting
monthly_sales.index = [f"{year}-{month:02d}" for year, month in monthly_sales.index]

plt.figure(figsize=(12,5))

plt.plot(
    monthly_sales.index,
    monthly_sales.values,
    marker='o',
    color='#4C72B0',
    linewidth=2
)

plt.title("Monthly Sales Trend", fontsize=14, fontweight='bold')
plt.xlabel("Month", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.xticks(rotation=45)
plt.grid(linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This line chart visualizes **total monthly sales** across the dataset, showing how revenue changed over time from 2020 to 2021. Each point represents the total sales generated in a specific month.

Features Used

- **Year** (categorical/time component)
- **Month** (categorical/time component)
- **Total Sales** (numerical, summed for each month)

Grouping Year and Month allows us to track sales performance over time.

Insight / Interpretation

The chart highlights two very different patterns for the two years:

2020 Trend

- Sales fluctuate throughout the year, with several ups and downs.
- Mid-2020 and late-2020 show **lower sales periods**, reflecting relatively weaker demand.

2021 Trend

- A major **surge in sales begins in January 2021**, indicating a strong shift in performance.
- Sales remain consistently higher compared to 2020.
- Significant peaks occur:
 May–July 2021 (mid-year boost)
 December 2021 (highest point, possibly seasonal demand)

Overall, the chart clearly shows **strong year-over-year growth**, with 2021 outperforming 2020 in almost every month.

What This Chart Shows

- Month-to-month sales fluctuations
- Seasonal or time-based patterns in sales
- Low-performing vs. high-performing months
- Strong growth trend from 2020 to 2021

This analysis helps identify periods of high demand, understand seasonal trends, and assess overall business growth over time.

Chart 16 – Quarterly Sales Trend (Line Chart)

Code:

```
#Chart 16 - Quarterly Sales Trend (Line Chart)

quarterly_sales = df.groupby(['Year', 'Quarter'])['Total Sales'].sum()

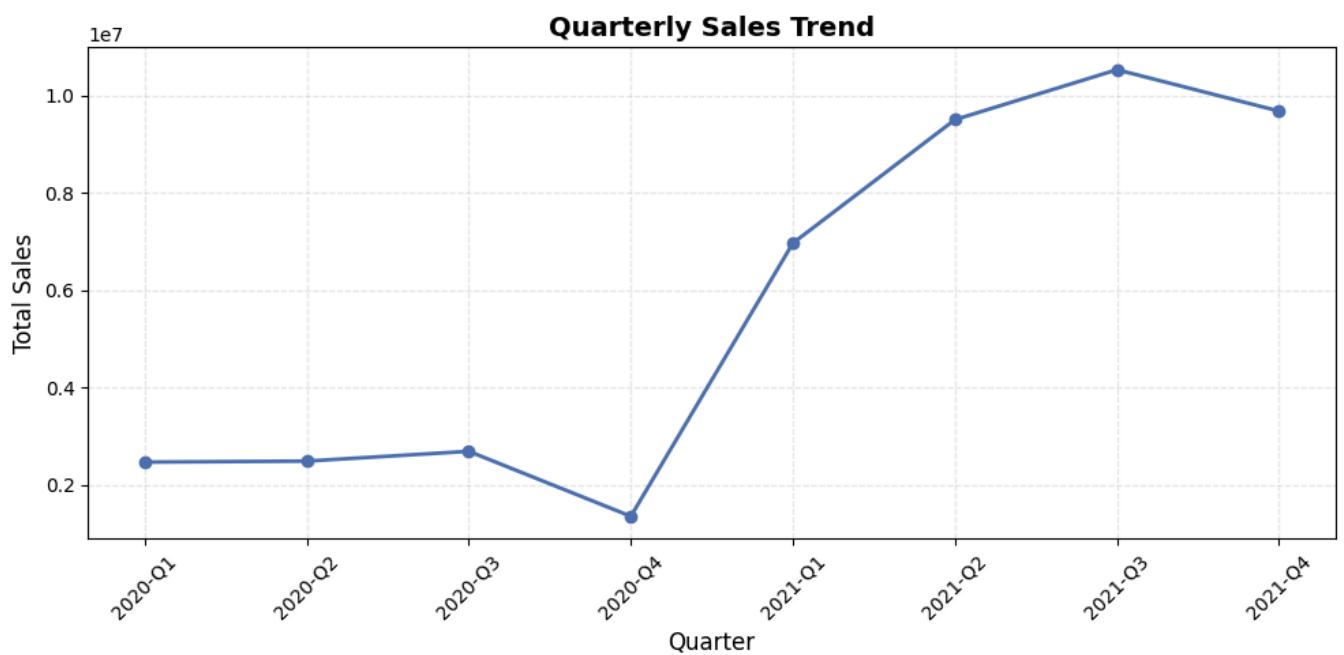
# Creating a proper index label 2021-Q1, 2021-Q2.
quarterly_sales.index = [f"{year}-Q{q}" for year, q in quarterly_sales.index]

plt.figure(figsize=(10,5))

plt.plot(
    quarterly_sales.index,
    quarterly_sales.values,
    marker='o',
    color='#4C72B0',
    linewidth=2
)

plt.title("Quarterly Sales Trend", fontsize=14, fontweight='bold')
plt.xlabel("Quarter", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.xticks(rotation=45)
plt.grid(linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:



Description

This line chart displays the **total sales for each quarter**, summarizing revenue every three months. This helps identify broader performance patterns and major shifts that may not be visible in month-to-month analysis.

Features Used

- **Year and Quarter** (time-based grouping)
- **Total Sales** (summed for each quarter)

These allow us to examine sales performance across eight quarters (2020–2021).

Insight / Interpretation

2020 Performance

- Sales remain **low and almost flat** across Q1–Q3 of 2020.
- **Q4 2020 declines sharply**, marking the weakest quarter in the dataset.
This indicates inconsistent and lower revenue in the first year.

2021 Performance

- A major improvement begins in **Q1 2021**, with sales rising sharply.
- **Q2 2021 and Q3 2021 continue the upward trend**, with **Q3 2021 becoming the highest-performing quarter overall**.
- **Q4 2021 dips slightly** from Q3 but still remains significantly higher than any quarter in 2020.

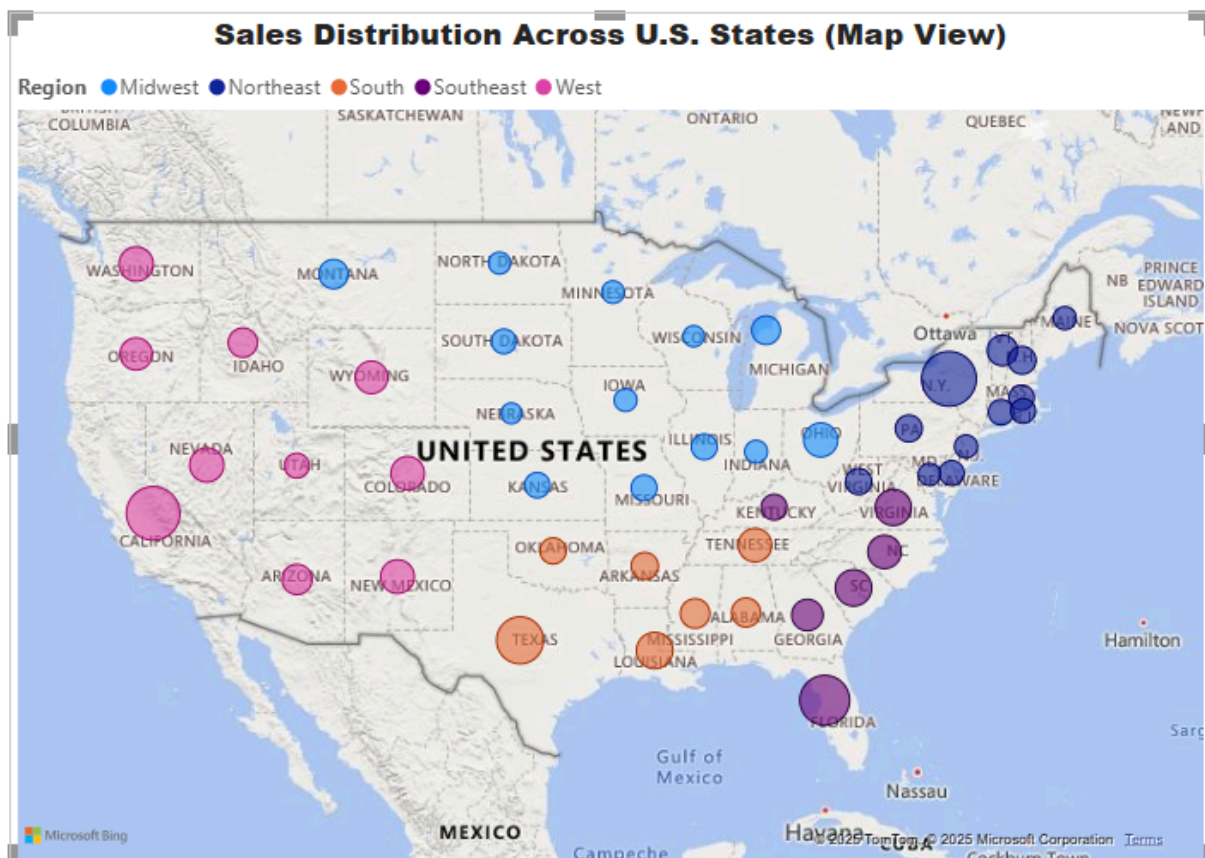
Overall, the chart shows a **strong and sustained growth trend throughout 2021**, outperforming 2020 in every quarter.

What This Chart Shows

- Quarter-to-quarter sales trends
- Strong vs. weak performing quarters
- A clear jump in revenue starting in 2021
- Whether there are seasonal patterns or yearly performance shifts

This visualization helps understand **long-term revenue patterns** and highlights how Adidas sales surged throughout 2021.

Geo Visual – Sales Distribution Across U.S. States (Map View)



Explanation of the Chart:

This map visual shows the total Adidas sales across U.S. states. Each bubble represents a state, with its size indicating the total sales amount and its color representing the region.

What the Chart Shows:

Larger bubbles appear in states such as California, Texas, New York, and Florida, indicating higher sales activity. Other states show moderate or lower sales contributions. Overall, the map highlights clear geographical patterns, showing where Adidas sales are stronger or weaker.

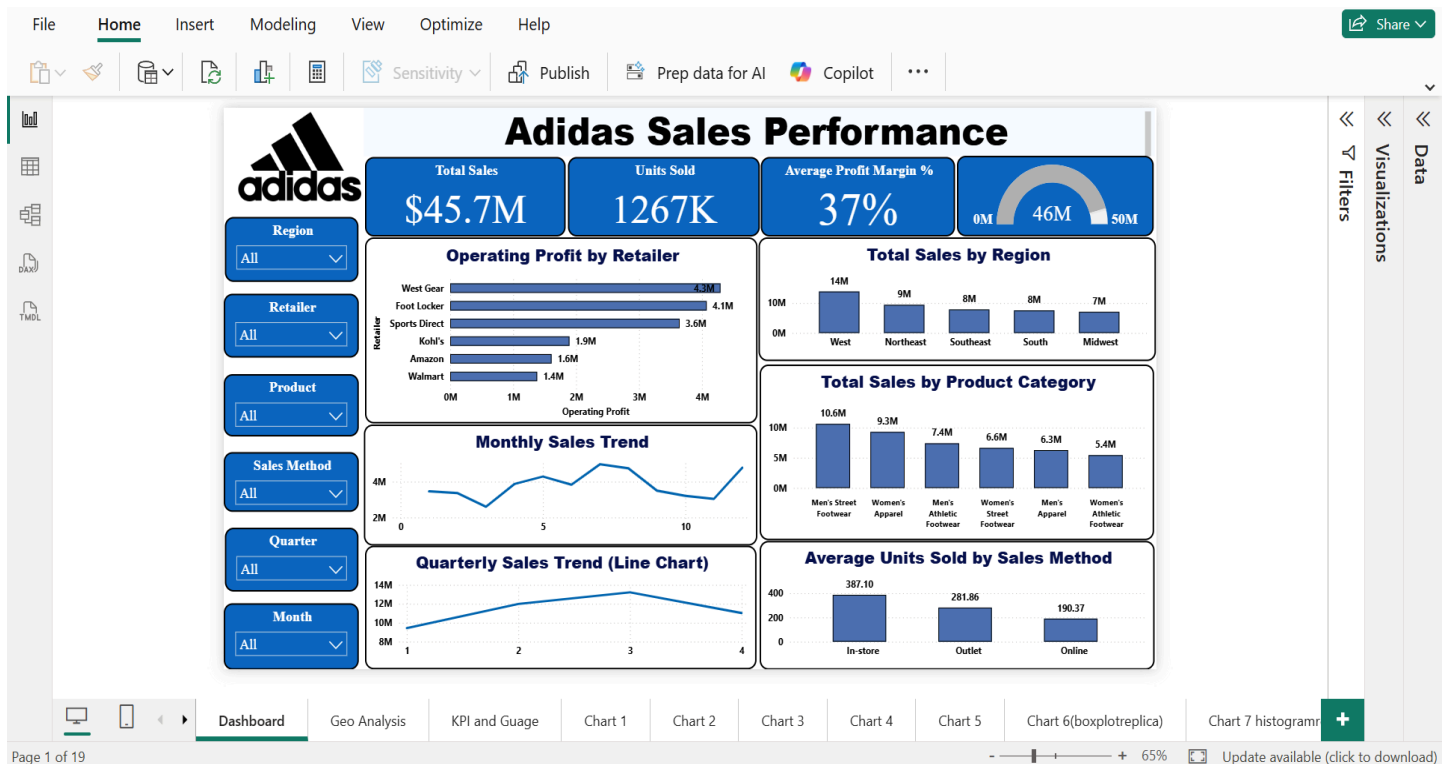
Features Used:

- **State** - geographic location
- **Region** - color category
- **Total Sales** - bubble size

What This Communicates:

The chart provides a geographic overview of sales performance, helping identify **high-performing states**, **regional trends**, and **potential areas for improvement or targeted strategies**.

Dashboard:



Dashboard Summary :

The **Adidas Sales Performance Dashboard** provides a consolidated overview of essential business metrics across **regions, retailers, product categories, and time periods**.

The top KPIs highlight strong overall performance:

- **Total Sales: \$45.7M**
- **Units Sold: 1.267M**
- **Average Profit Margin: 37%**
- **Gauge Chart – Operating Profit:**

A gauge visual displays the **overall operating profit**, clearly showing progress toward the performance target.

Interactive slicers (**Region, Retailer, Product, Sales Method, Quarter, Month**) allow users to **filter the dashboard** and explore the data from multiple perspectives.

Key visual insights include:

- **Operating Profit by Retailer:**
West Gear and Foot Locker generate the highest profit, while Amazon and Walmart contribute the least.
- **Total Sales by Region:**
The West and Northeast regions lead in revenue, showing stronger market presence.
- **Product Category Sales:**
Footwear, especially Men's Street Footwear, is the top revenue-generating category.

- **Monthly and Quarterly Sales Trends:**

- **2020:** Lower, fluctuating sales
- **2021:** Significant growth, with peaks in **Q3** and **December**
These trends highlight strong seasonal and annual performance shifts.

- **Average Units Sold by Sales Method:**

In-store customers purchase more units per transaction, while **online buyers** tend to make smaller purchases.

Overall, the dashboard presents a clear story of:

- **Strong recovery and growth in 2021**
- **Footwear as the primary revenue driver**
- **Consistently high-performing regions and retailers**
- Insights supporting **better regional planning, retailer strategy, inventory allocation, and targeted marketing efforts.**

Insights Generated:

1. The West and Northeast regions generate the highest sales.

Both charts and the dashboard show that the **West** is the strongest market, followed by the **Northeast**. These regions have higher customer activity and larger revenue. The **Midwest and Southeast** perform much lower in comparison.

2. Footwear is the top-performing product category.

Men's and Women's footwear categories generate much higher revenue than apparel. **Men's street footwear, in particular**, is one of the biggest contributors. This shows that footwear drives most of Adidas' business in this dataset.

3. West Gear and Foot Locker are the most profitable retailers.

Retail analysis shows that **West Gear** and **Foot Locker** bring the highest operating profit. Retailers like **Amazon** and **Walmart** show much lower profits, even when they have reasonable sales volume. This highlights important differences in retailer performance.

4. Sales significantly improved in 2021 compared to 2020.

2020 had several low and unstable months. But in **2021**, sales increased sharply, especially around **Q3** and **December**. This shows a strong recovery and growing demand in the later period.

5. In-store purchases have the highest units sold per transaction.

Customers who buy in-store tend to purchase **more items at once**, while online orders usually contain only one or two items. This means offline stores encourage bigger baskets, possibly due to store displays or sales assistance.

6. Sales follow seasonal patterns with clear peak months.

Sales rise noticeably during **May–July** and in **December**. These peaks suggest seasonal demand such as mid-year shopping and holiday sales, helping explain when customers tend to buy more.

7. Units Sold has a strong impact on revenue and profit.

The correlation heatmap shows a strong positive relationship between **Units Sold, Total Sales, and Operating Profit**. This means selling more quantity directly increases sales and profit, which is expected in retail.

8. Outliers in sales and profit represent real high-value transactions.

There are many high values for Units Sold, Total Sales, and Operating Profit, but these are **not errors**. They represent **bulk orders or premium product sales**, which naturally create spikes and lead to right-skewed distributions.

9. Profit margins are different across retailers and product types.

Some retailers achieve high revenue but lower profit margins, while others bring in higher margins even with fewer transactions. This difference can be due to pricing strategies, discount levels, or customer segments.

10. Online sales happen frequently but involve fewer items per order.

Online is the most commonly used sales channel, but each order usually contains smaller quantities. In-store sales happen less often but contribute more units per transaction. This shows both channels have different customer behaviours.

Future Enhancements:

- **Add more years of data** so that long-term sales forecasting and seasonal patterns can be studied more accurately.
- **Include customer-level information** (age, gender, loyalty, purchase frequency) to build deeper insights and personalised recommendations.
- **Add advanced product attributes** (model, colour, category type) to understand which specific features drive sales.
- **Develop predictive models** in Python (using ARIMA/Prophet/Regression) to forecast future sales and assist inventory planning.
- **Automate the entire workflow** by scheduling Python scripts for cleaning and enabling Power BI's scheduled data refresh for real-time dashboards.
- **Integrate external datasets** (economic indicators, holiday periods, marketing spend) to understand external factors affecting sales.
- **Enhance dashboard features** with drill-through pages, tooltip summaries, and comparative views for deeper business insights.

Conclusion:

The Adidas sales dataset initially contained inconsistent formats, missing values, and mixed data types, which made direct analysis difficult. These issues were resolved through systematic data cleaning and preprocessing in Python, where numerical fields were corrected, missing values were handled appropriately, date-based features were extracted, and a new Profit Margin % metric was created to support deeper analysis.

Through visualisations and dashboard exploration, the project conveyed several important business insights, including regional sales performance, retailer profitability, product category strength, customer buying behaviour, and clear trends across months and quarters. These insights helped identify where Adidas performs strongly, where sales are lower, which retailers contribute most to profit, and which product categories drive overall revenue.

By solving these analytical questions, the project provides valuable guidance for strategic decisions such as marketing focus, retailer partnerships, product planning, and inventory allocation. For example, regions and products with high demand can be prioritised for stocking, while peak sales periods like Q3 and December can be supported with appropriate inventory levels. The interactive Power BI dashboard brings all these insights together in a simple, user-friendly format, allowing decision-makers to explore the data and take actions based on clear evidence.